

kryzotech

Python 

Table of Content:

- Introduction
- Installation
- First Program
- Variable
- Type Conversion
- Python String
- Print Function
- Python Booleans
- Python operator
- Comment
- Python Input
- Python Collection(List, Tuple, Set, Dictionary)
- If...Else
- Python Loop(While,For)

Table of Content:

- Python Statements
- Range Function
- Function
- Lambda
- Array
- Math Function
- Class
- Inheritance
- Iterators
- Exception
- Project

1. Introduction:

Python is an interpreted **high-level general-purpose programming language**. Its design philosophy emphasizes code readability with its use of significant indentation. Its language constructs as well as its object-oriented approach aim to help programmers write clear, logical code for small and large-scale projects.

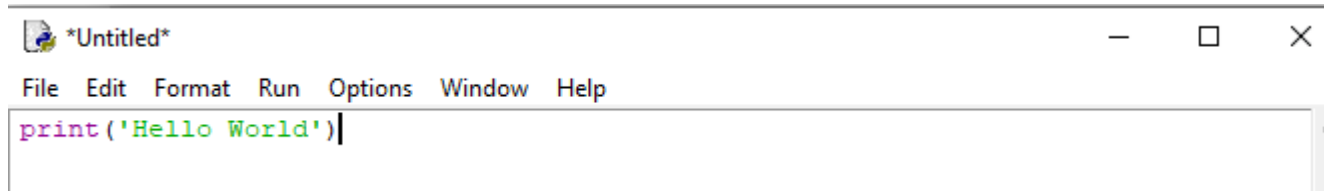
2. Installation:

Live

3.First program:

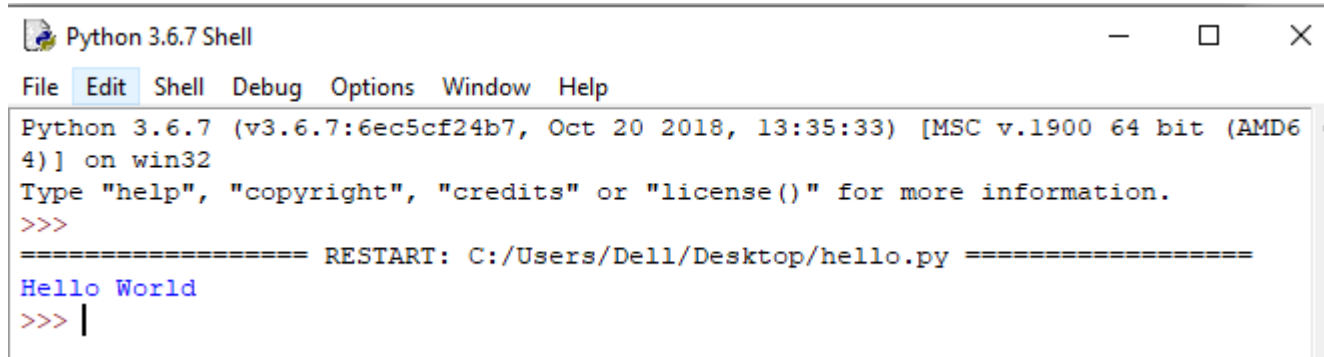
The print() function prints the specified message to the screen, or other standard output device.

Input:



```
*Untitled*  
File Edit Format Run Options Window Help  
print('Hello World')
```

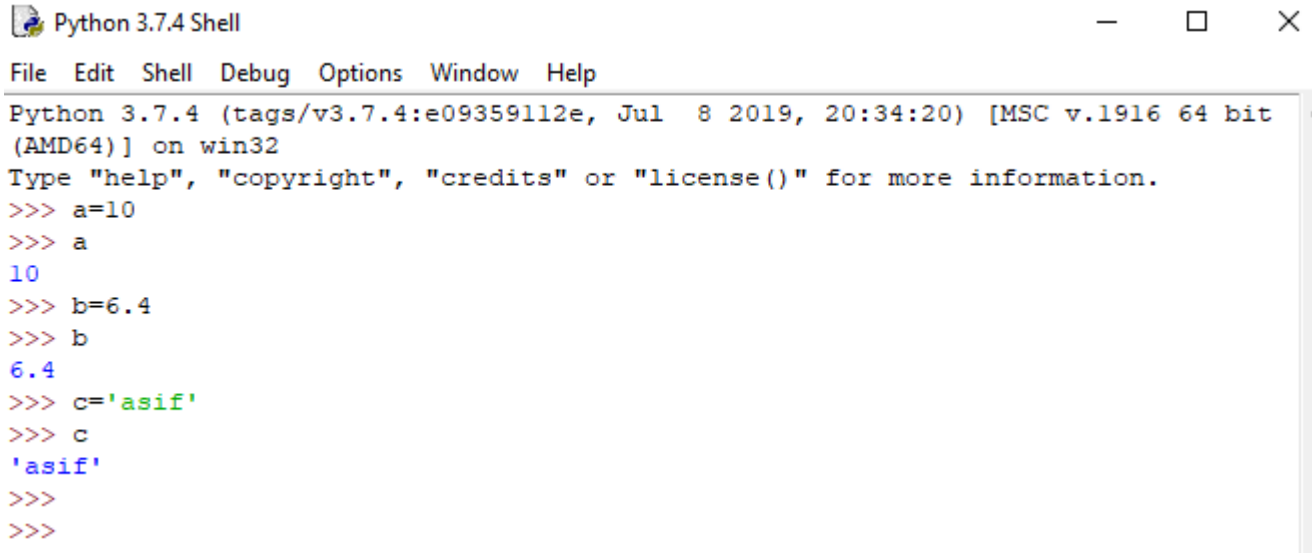
Output:



```
Python 3.6.7 Shell  
File Edit Shell Debug Options Window Help  
Python 3.6.7 (v3.6.7:6ec5cf24b7, Oct 20 2018, 13:35:33) [MSC v.1900 64 bit (AMD64)] on win32  
Type "help", "copyright", "credits" or "license()" for more information.  
>>>  
===== RESTART: C:/Users/Dell/Desktop/hello.py =====  
Hello World  
>>> |
```

4. Variable:

A Python variable is a reserved memory location to store values. In other words, a variable in a python program gives data to the computer for processing.



```
Python 3.7.4 Shell
File Edit Shell Debug Options Window Help
Python 3.7.4 (tags/v3.7.4:e09359112e, Jul  8 2019, 20:34:20) [MSC v.1916 64 bit
(AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> a=10
>>> a
10
>>> b=6.4
>>> b
6.4
>>> c='asif'
>>> c
'asif'
>>>
>>>
```

Data type:

Text Type: `str`

Numeric Types: `int`, `float`, `complex`

Sequence Types: `list`, `tuple`, `range`

Mapping Type: `dict`

Set Types: `set`, `frozenset`

Boolean Type: `bool`

Binary Types: `bytes`, `bytearray`, `memoryview`

Numeric data type:

There are three numeric types in Python:

- Int
- Float
- Complex

```
a=10 #int  
b=3.8 #float  
c=12j #complex
```

5.Type conversion:

Python casting:

Int:

```
a=int(5)    # a will be 5  
b=int(3.8)  # b will be 3  
c=int("12") # c will be 12
```

Float:

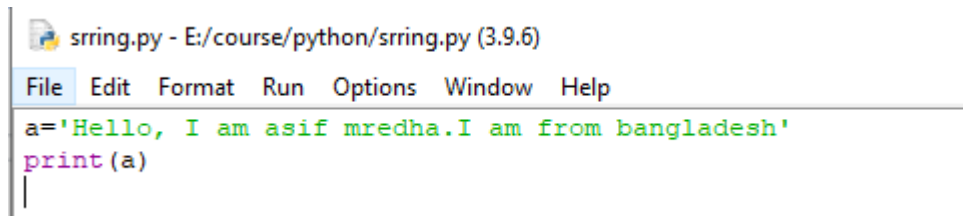
```
x=float(5)  # a will be 5.0  
y=float(3.8) # b will be 3.8  
z=float("12") # c will be 12.0  
w=float("45.7") # c will be 45.7
```

String:

```
a=str("5") # a will be "5"  
b=str(3.8) # b will be "3.8"  
c=str("a1") # c will be "a1"  
d=str("T2") # c will be "T2"
```

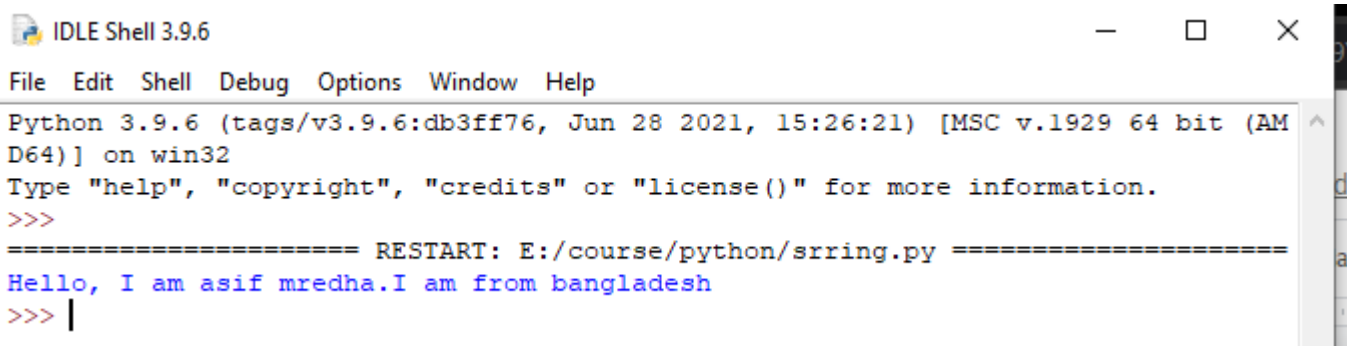
6. Python String:

Input:



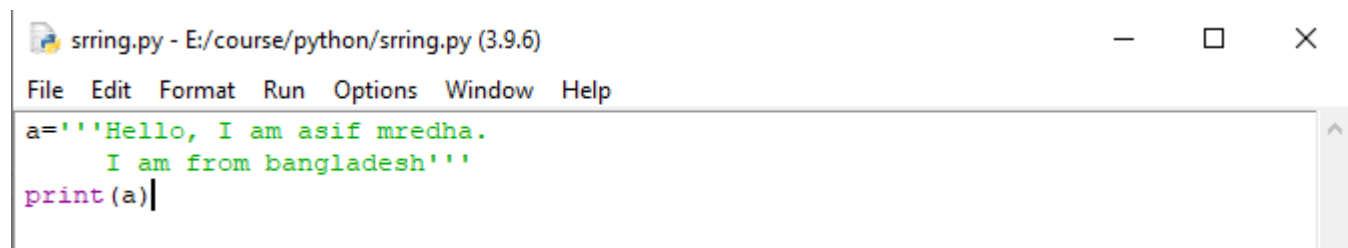
```
srring.py - E:/course/python/srring.py (3.9.6)
File Edit Format Run Options Window Help
a='Hello, I am asif mredha.I am from bangladesh'
print(a)
|
```

Output:



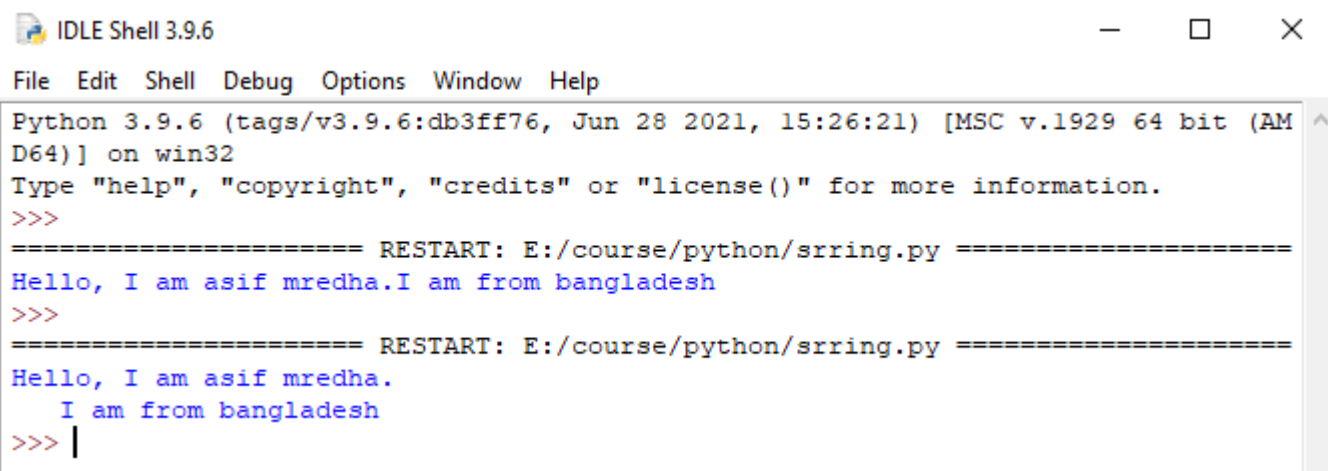
```
IDLE Shell 3.9.6
File Edit Shell Debug Options Window Help
Python 3.9.6 (tags/v3.9.6:db3ff76, Jun 28 2021, 15:26:21) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: E:/course/python/srring.py =====
Hello, I am asif mredha.I am from bangladesh
>>> |
```

Input:



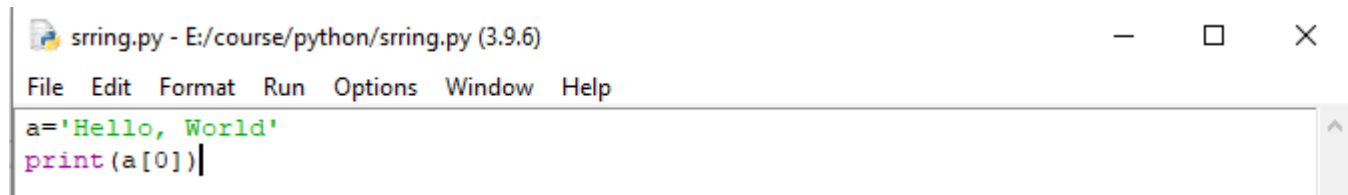
```
srring.py - E:/course/python/srring.py (3.9.6)
File Edit Format Run Options Window Help
a='''Hello, I am asif mredha.
    I am from bangladesh'''
print(a)
```

Output:



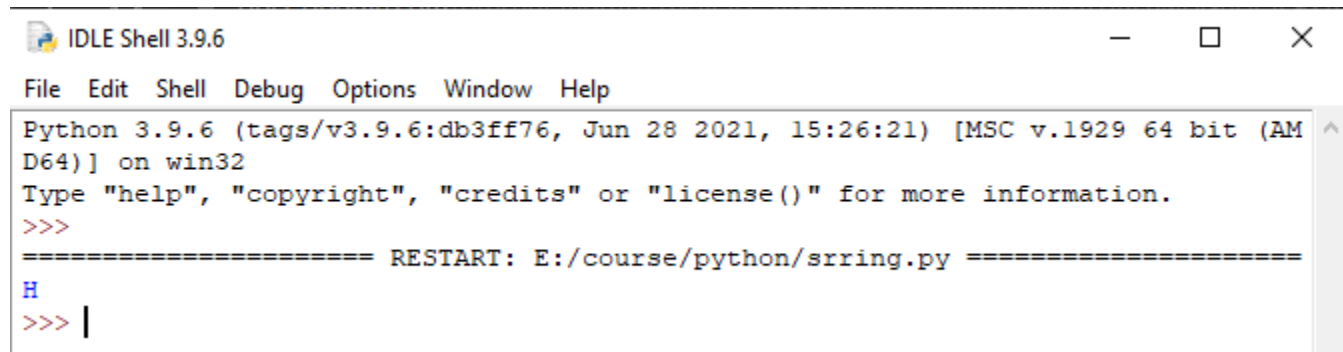
```
IDLE Shell 3.9.6
File Edit Shell Debug Options Window Help
Python 3.9.6 (tags/v3.9.6:db3ff76, Jun 28 2021, 15:26:21) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: E:/course/python/srring.py =====
Hello, I am asif mredha.I am from bangladesh
>>>
===== RESTART: E:/course/python/srring.py =====
Hello, I am asif mredha.
    I am from bangladesh
>>> |
```

Input:

A screenshot of a Python script file named 'srring.py' located at 'E:/course/python/srring.py' using Python 3.9.6. The window has a standard title bar with minimize, maximize, and close buttons. The menu bar includes 'File', 'Edit', 'Format', 'Run', 'Options', 'Window', and 'Help'. The code editor shows two lines of Python code: `a='Hello, World'` and `print(a[0])`.

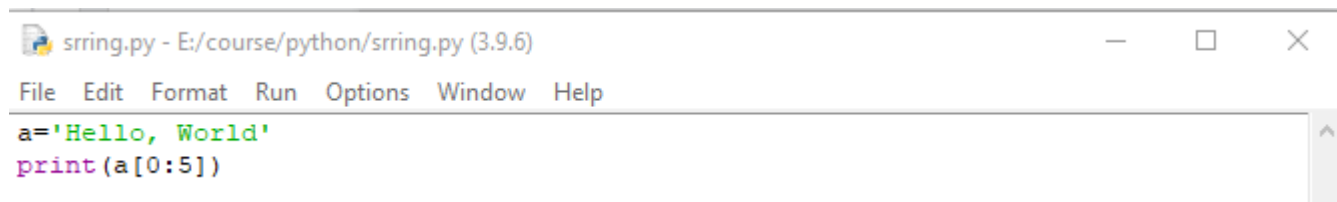
```
srring.py - E:/course/python/srring.py (3.9.6)
File Edit Format Run Options Window Help
a='Hello, World'
print(a[0])
```

Output:

A screenshot of the 'IDLE Shell 3.9.6' window. The title bar shows the icon, name, and standard window controls. The menu bar includes 'File', 'Edit', 'Shell', 'Debug', 'Options', 'Window', and 'Help'. The shell displays the Python 3.9.6 startup message, including version tags and build information. It prompts the user to type 'help', 'copyright', 'credits', or 'license()' for more information. After the prompt, a separator line is shown, followed by the command 'RESTART: E:/course/python/srring.py'. Below this, the character 'H' is entered at the prompt, and the shell is ready for the next input.

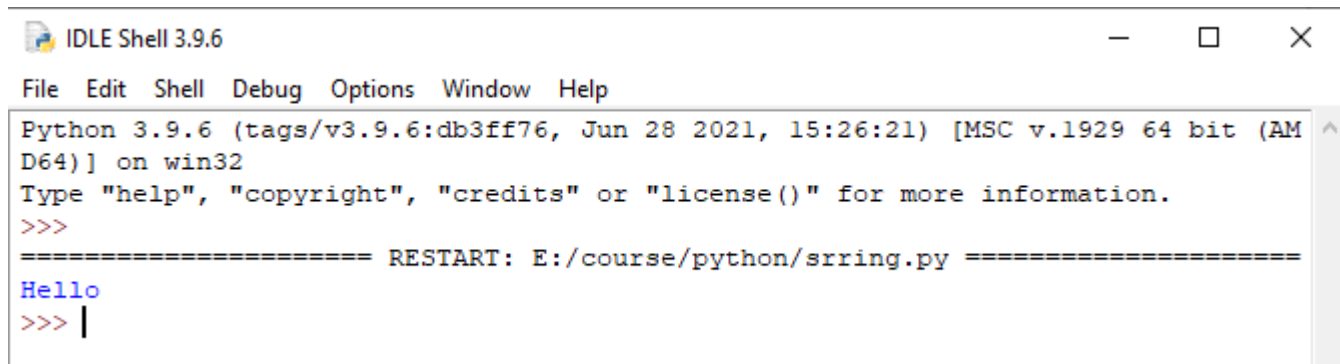
```
IDLE Shell 3.9.6
File Edit Shell Debug Options Window Help
Python 3.9.6 (tags/v3.9.6:db3ff76, Jun 28 2021, 15:26:21) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: E:/course/python/srring.py =====
H
>>> |
```

Input:



```
srring.py - E:/course/python/srring.py (3.9.6)
File Edit Format Run Options Window Help
a='Hello, World'
print(a[0:5])
```

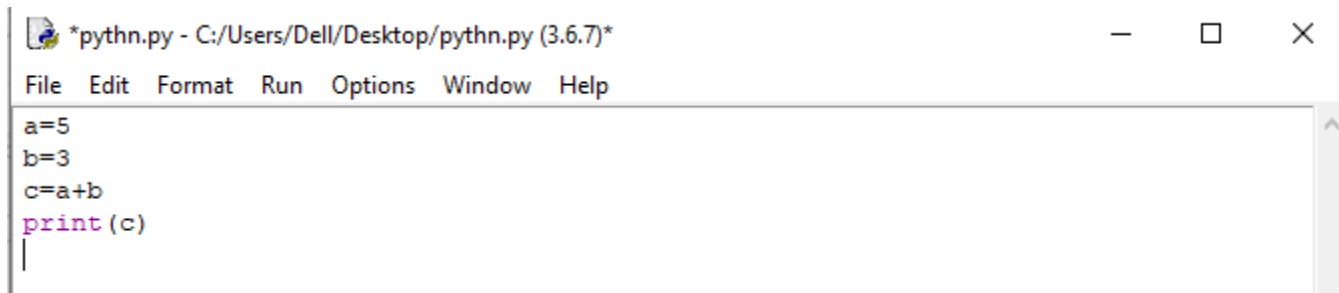
Output:



```
IDLE Shell 3.9.6
File Edit Shell Debug Options Window Help
Python 3.9.6 (tags/v3.9.6:db3ff76, Jun 28 2021, 15:26:21) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: E:/course/python/srring.py =====
Hello
>>> |
```

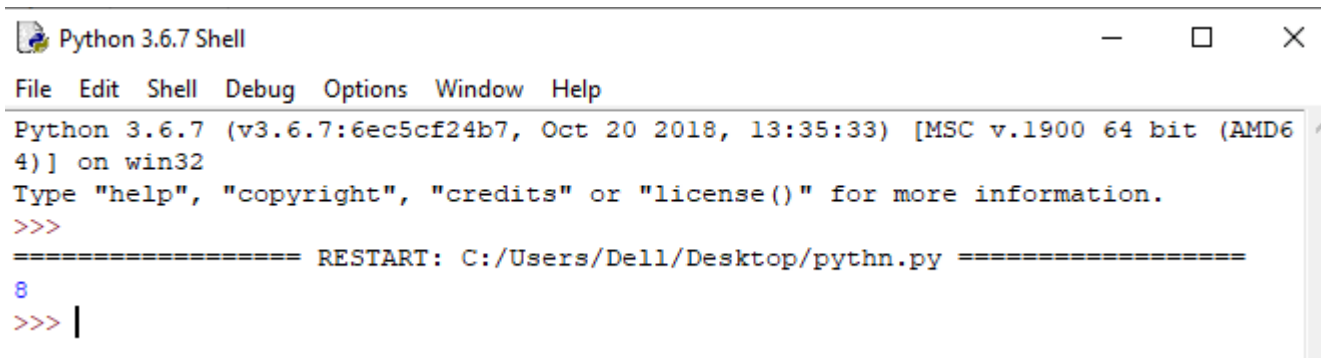
7. Print Function:

Input:



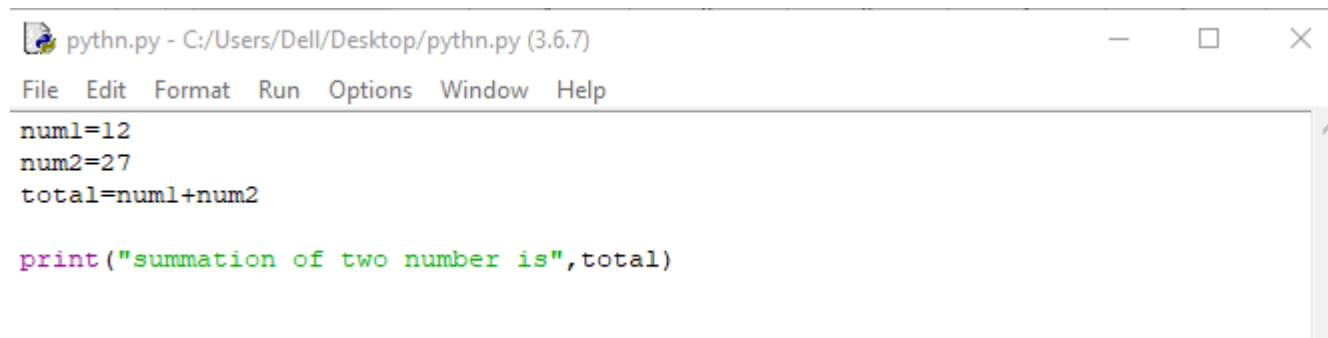
```
*pythn.py - C:/Users/Dell/Desktop/pythn.py (3.6.7)*
File Edit Format Run Options Window Help
a=5
b=3
c=a+b
print(c)
|
```

Output:



```
Python 3.6.7 Shell
File Edit Shell Debug Options Window Help
Python 3.6.7 (v3.6.7:6ec5cf24b7, Oct 20 2018, 13:35:33) [MSC v.1900 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/Dell/Desktop/pythn.py =====
8
>>> |
```

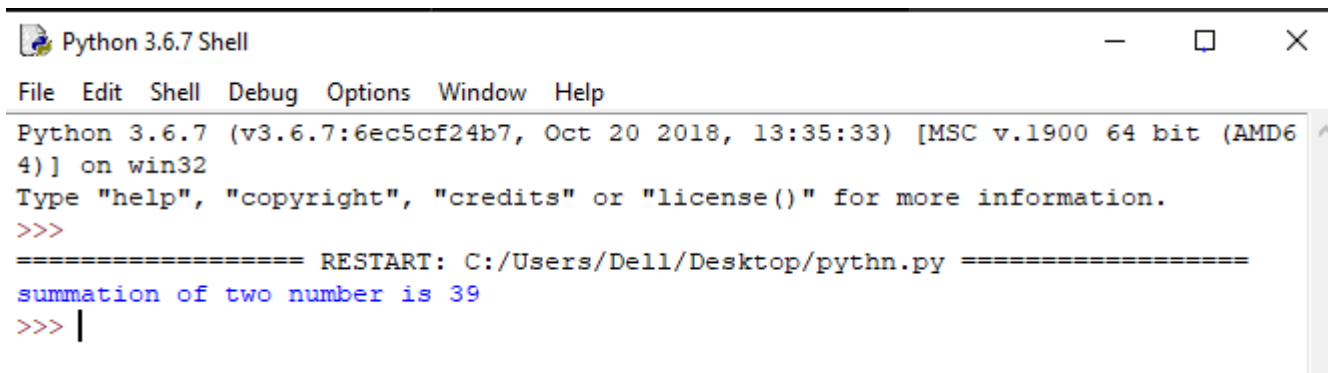
Input:



```
pythn.py - C:/Users/Dell/Desktop/pythn.py (3.6.7)
File Edit Format Run Options Window Help
num1=12
num2=27
total=num1+num2

print("summation of two number is",total)
```

Output:



```
Python 3.6.7 Shell
File Edit Shell Debug Options Window Help
Python 3.6.7 (v3.6.7:6ec5cf24b7, Oct 20 2018, 13:35:33) [MSC v.1900 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/Dell/Desktop/pythn.py =====
summation of two number is 39
>>> |
```

Input:

```
num1=12
num2=27
total=num1+num2

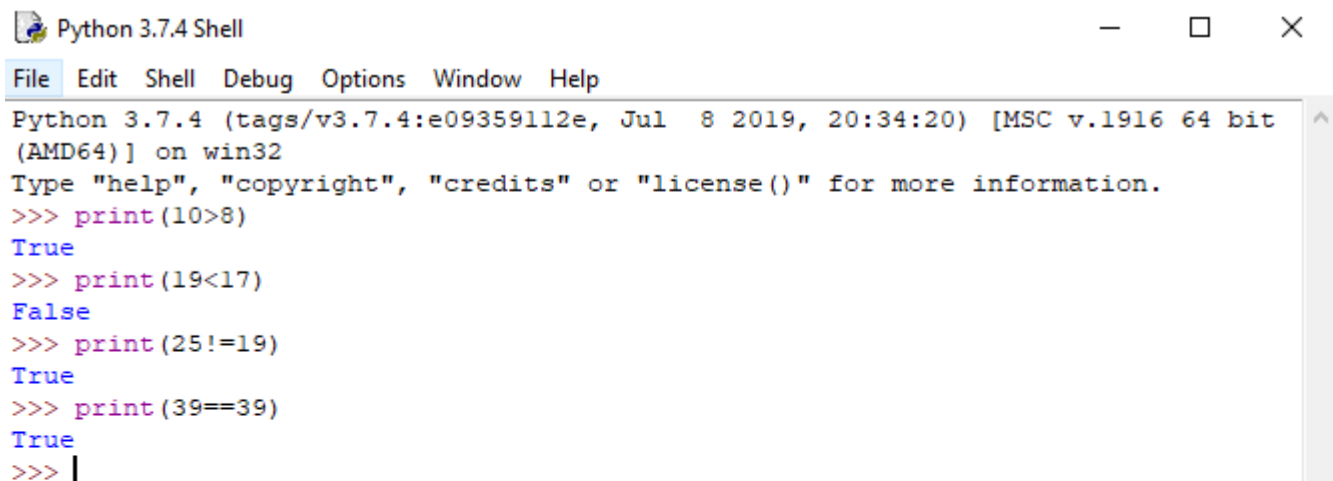
print("summation of",num1,"and",num2,"is",total)
```

Output:

```
Python 3.10.2 (tags/v3.10.2:a58ebcc, Jan 17 2022, 14:12:15) [MSC v.1929 64 b
AMD64] on win32
Type "help", "copyright", "credits" or "license()" for more information.

===== RESTART: D:/asif-office-work/python_code/pythn.py =====
summation of 12 and 27 is 39
```

8.Python booleans:



```
Python 3.7.4 Shell
File Edit Shell Debug Options Window Help
Python 3.7.4 (tags/v3.7.4:e09359112e, Jul 8 2019, 20:34:20) [MSC v.1916 64 bit
(AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> print(10>8)
True
>>> print(19<17)
False
>>> print(25!=19)
True
>>> print(39==39)
True
>>> |
```

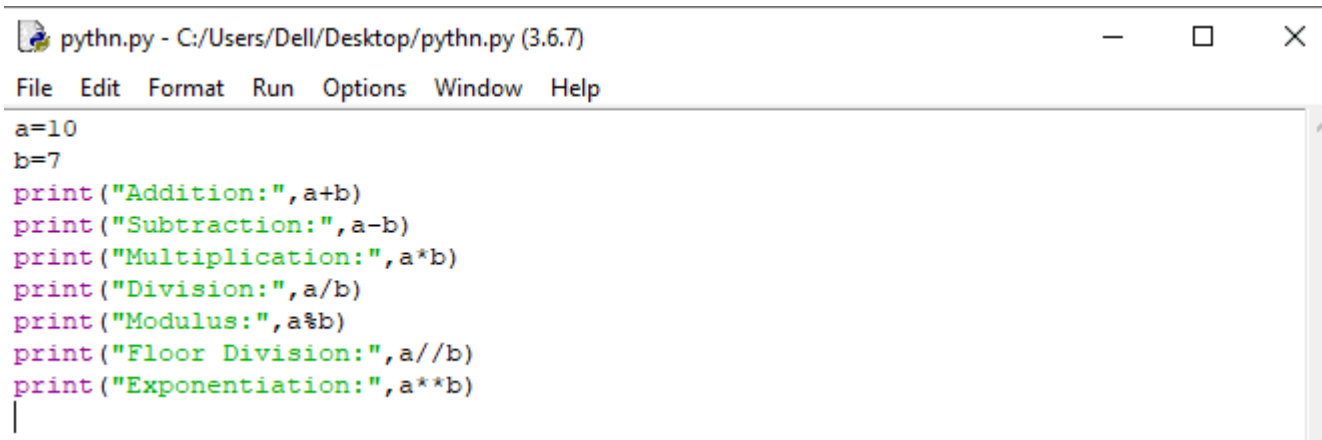
9.Python Operator:

Operators are used to perform operations on variables.

- Arithmetic Operators
- Assignment Operators
- Comparison Operators
- Logical Operators
- Identity Operators
- Membership Operators
- Bitwise Operators

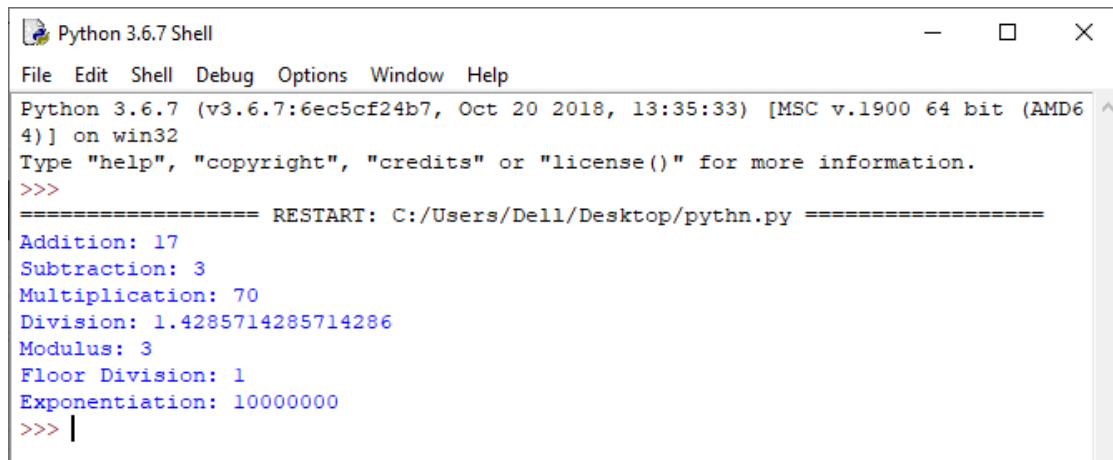
- Arithmetic Operator:

Input:



```
pythn.py - C:/Users/Dell/Desktop/pythn.py (3.6.7)
File Edit Format Run Options Window Help
a=10
b=7
print("Addition:",a+b)
print("Subtraction:",a-b)
print("Multiplication:",a*b)
print("Division:",a/b)
print("Modulus:",a%b)
print("Floor Division:",a//b)
print("Exponentiation:",a**b)
```

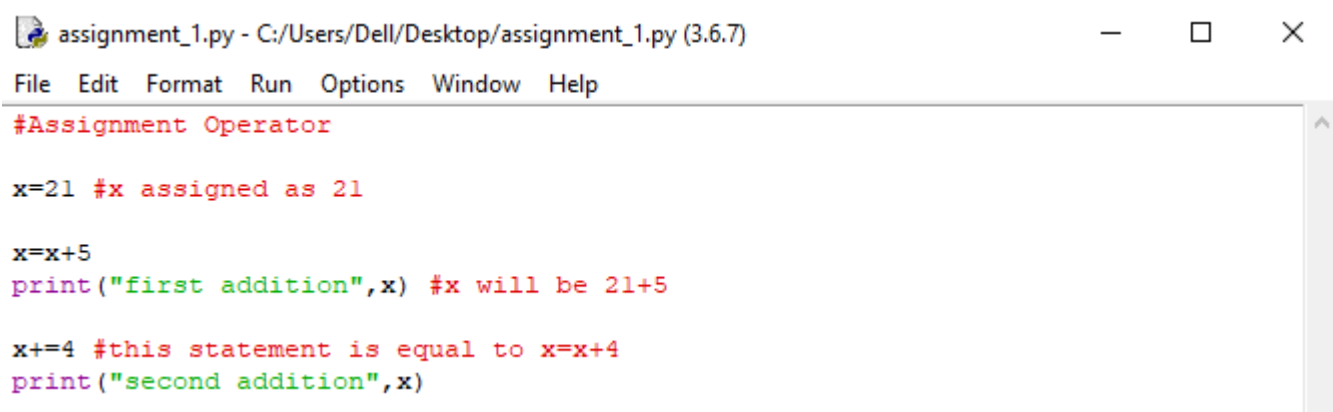
Output:



```
Python 3.6.7 Shell
File Edit Shell Debug Options Window Help
Python 3.6.7 (v3.6.7:6ec5cf24b7, Oct 20 2018, 13:35:33) [MSC v.1900 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/Dell/Desktop/pythn.py =====
Addition: 17
Subtraction: 3
Multiplication: 70
Division: 1.4285714285714286
Modulus: 3
Floor Division: 1
Exponentiation: 10000000
>>>
```

- Assignment Operator:

Input:



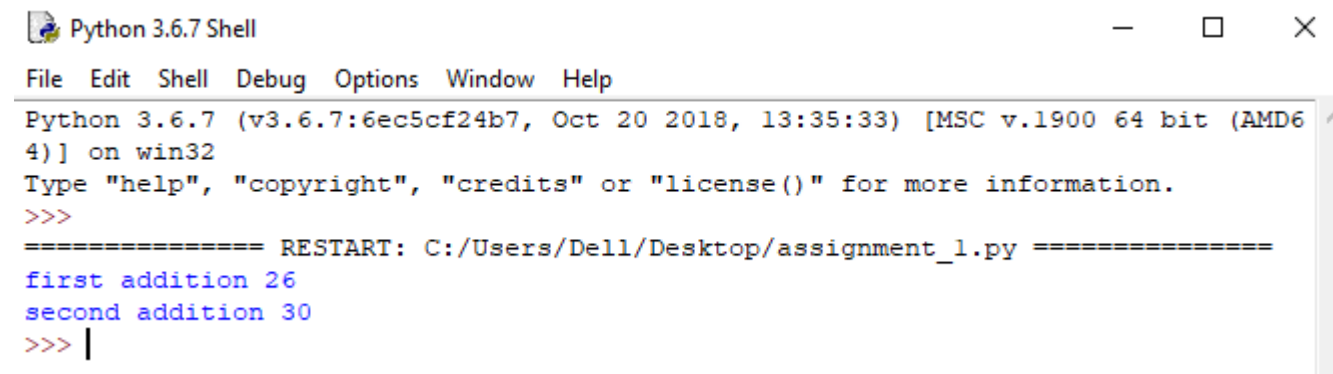
```
assignment_1.py - C:/Users/Dell/Desktop/assignment_1.py (3.6.7)
File Edit Format Run Options Window Help
#Assignment Operator

x=21 #x assigned as 21

x=x+5
print("first addition",x) #x will be 21+5

x+=4 #this statement is equal to x=x+4
print("second addition",x)
```

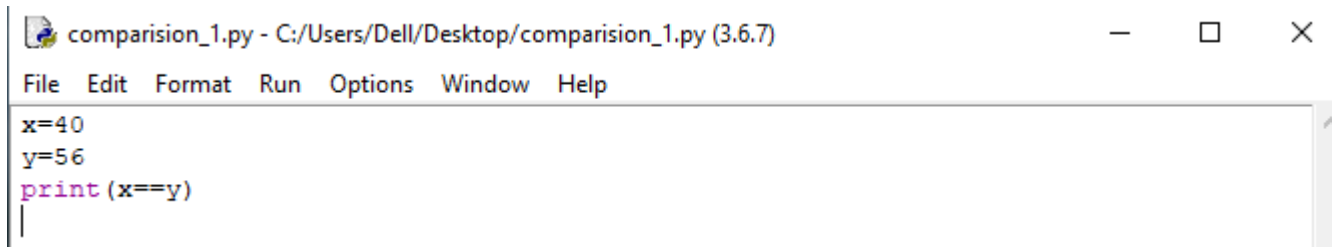
Output:



```
Python 3.6.7 Shell
File Edit Shell Debug Options Window Help
Python 3.6.7 (v3.6.7:6ec5cf24b7, Oct 20 2018, 13:35:33) [MSC v.1900 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/Dell/Desktop/assignment_1.py =====
first addition 26
second addition 30
>>> |
```

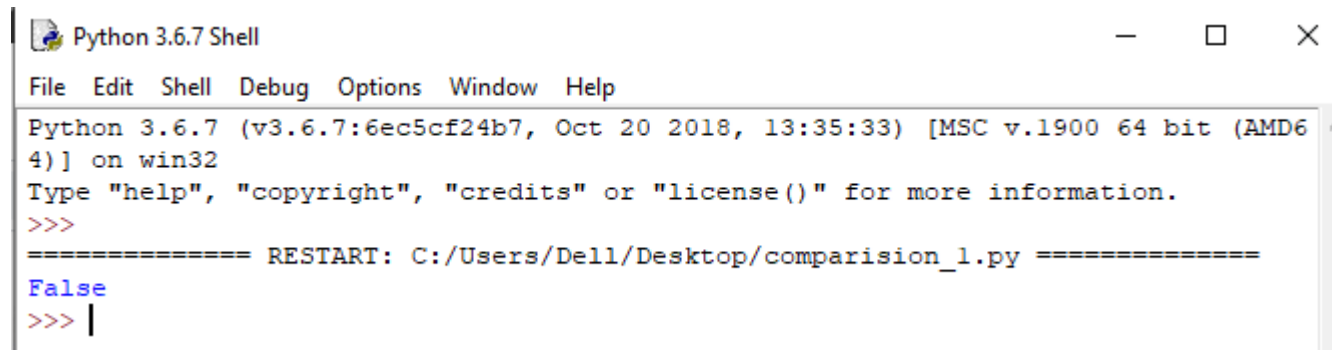
- **Comparison Operator:**

Input:



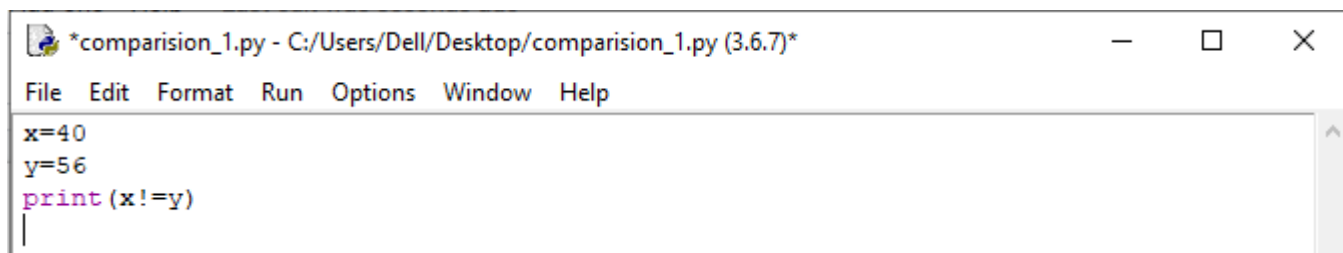
```
comparision_1.py - C:/Users/Dell/Desktop/comparision_1.py (3.6.7)
File Edit Format Run Options Window Help
x=40
y=56
print(x==y)
|
```

Output:



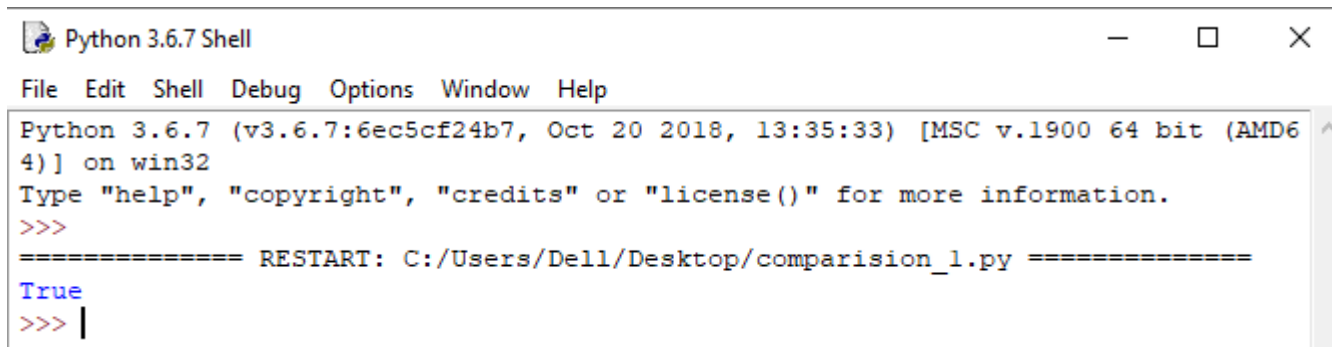
```
Python 3.6.7 Shell
File Edit Shell Debug Options Window Help
Python 3.6.7 (v3.6.7:6ec5cf24b7, Oct 20 2018, 13:35:33) [MSC v.1900 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/Dell/Desktop/comparision_1.py =====
False
>>> |
```

Input:



```
*comparision_1.py - C:/Users/Dell/Desktop/comparision_1.py (3.6.7)*
File Edit Format Run Options Window Help
x=40
y=56
print(x!=y)
|
```

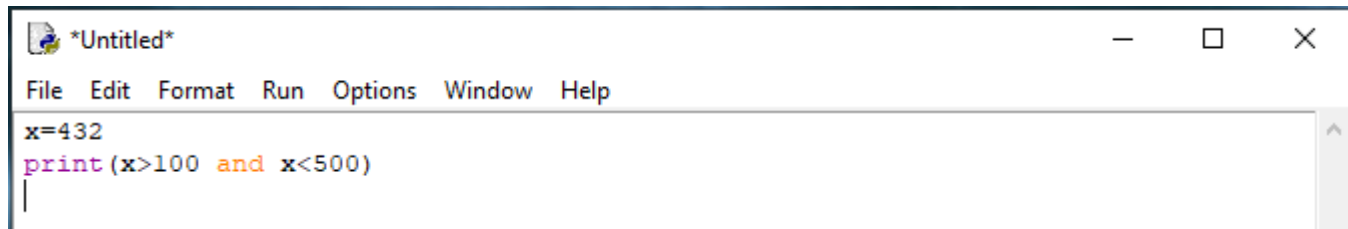
Output:



```
Python 3.6.7 Shell
File Edit Shell Debug Options Window Help
Python 3.6.7 (v3.6.7:6ec5cf24b7, Oct 20 2018, 13:35:33) [MSC v.1900 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/Dell/Desktop/comparision_1.py =====
True
>>> |
```

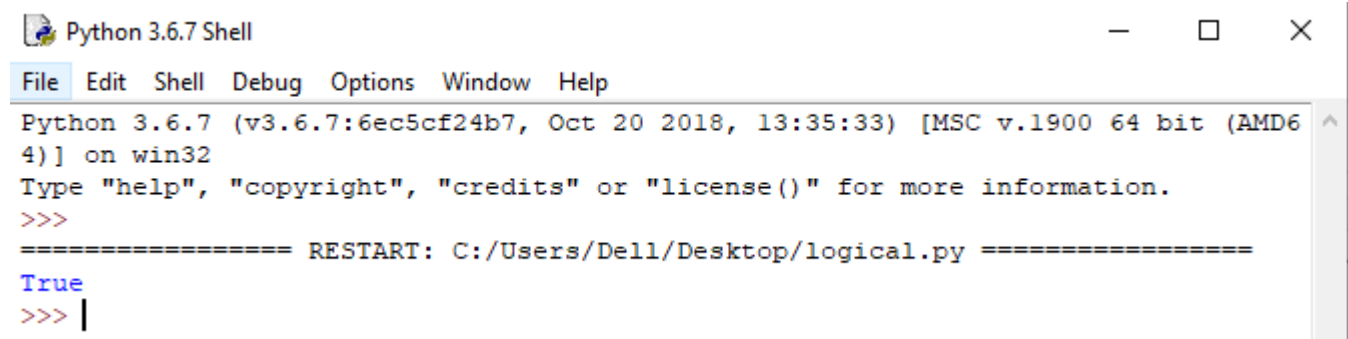
- Logical Operator:

Input:



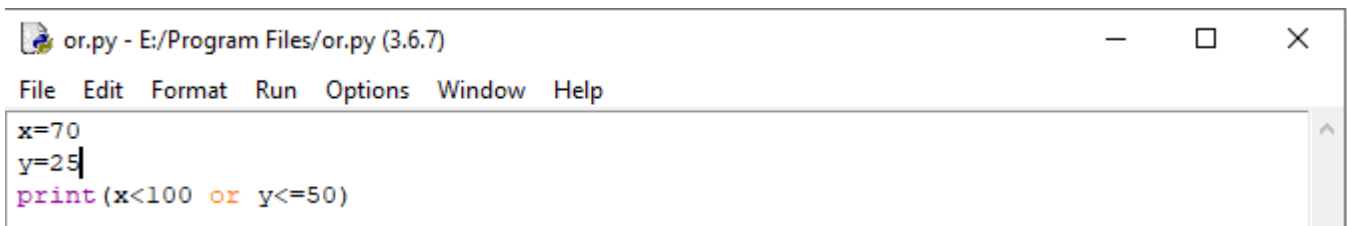
```
*Untitled*
File Edit Format Run Options Window Help
x=432
print(x>100 and x<500)
|
```

Output:



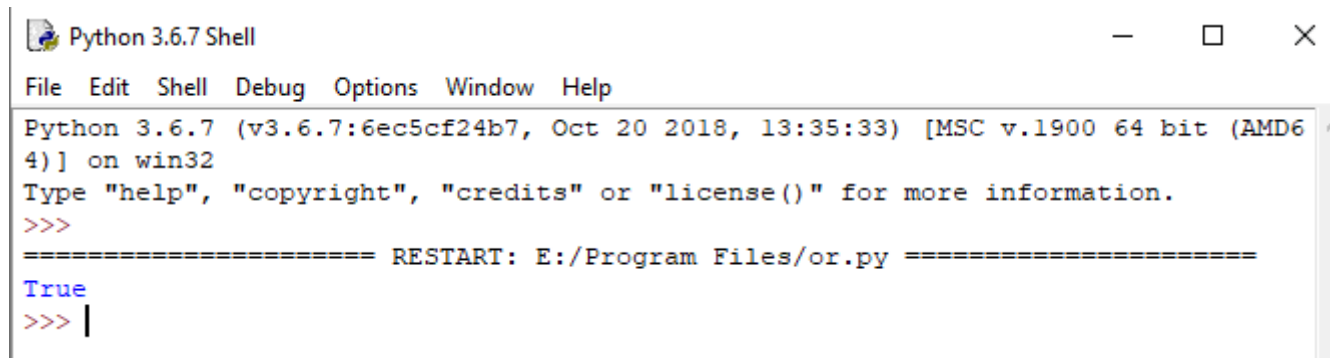
```
Python 3.6.7 Shell
File Edit Shell Debug Options Window Help
Python 3.6.7 (v3.6.7:6ec5cf24b7, Oct 20 2018, 13:35:33) [MSC v.1900 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/Dell/Desktop/logical.py =====
True
>>> |
```

Input:



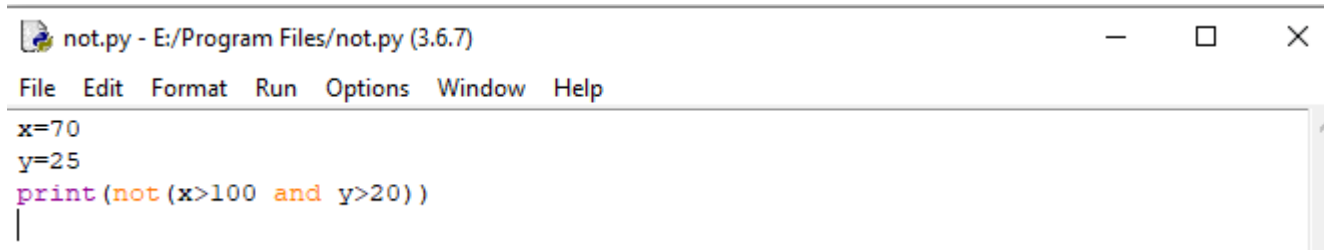
```
or.py - E:/Program Files/or.py (3.6.7)
File Edit Format Run Options Window Help
x=70
y=25
print(x<100 or y<=50)
```

Output:



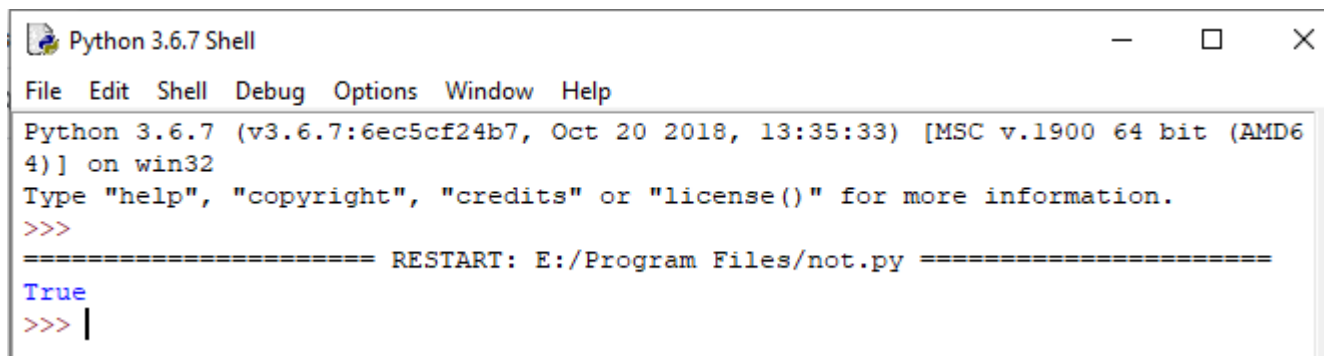
```
Python 3.6.7 Shell
File Edit Shell Debug Options Window Help
Python 3.6.7 (v3.6.7:6ec5cf24b7, Oct 20 2018, 13:35:33) [MSC v.1900 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: E:/Program Files/or.py =====
True
>>> |
```

Input:



```
not.py - E:/Program Files/not.py (3.6.7)
File Edit Format Run Options Window Help
x=70
y=25
print(not(x>100 and y>20))
|
```

Output:

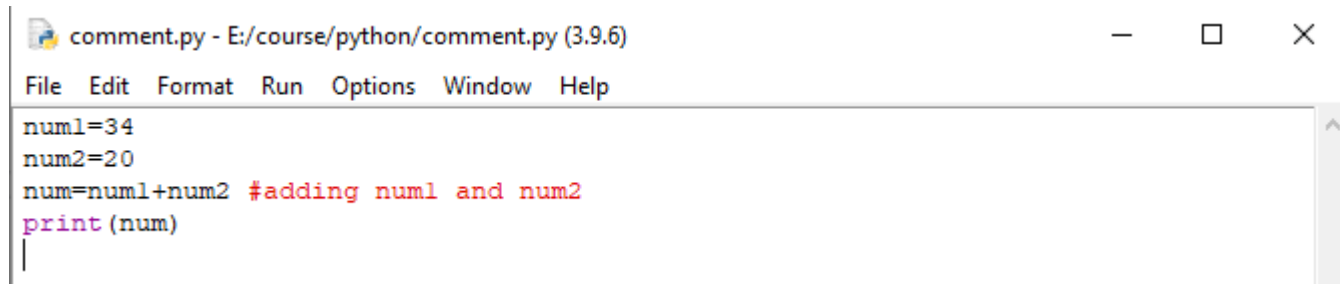


```
Python 3.6.7 Shell
File Edit Shell Debug Options Window Help
Python 3.6.7 (v3.6.7:6ec5cf24b7, Oct 20 2018, 13:35:33) [MSC v.1900 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: E:/Program Files/not.py =====
True
>>> |
```

10. Comments:

- A Python comment is a line of text in a program that is not executed by the interpreter. Comments are used during debugging to identify issues and to explain code.
- A Single line comment in Python starts with the hash character # and extends to the end of the physical line.
- A Multiline comment in Python can be created using triple-quoted strings

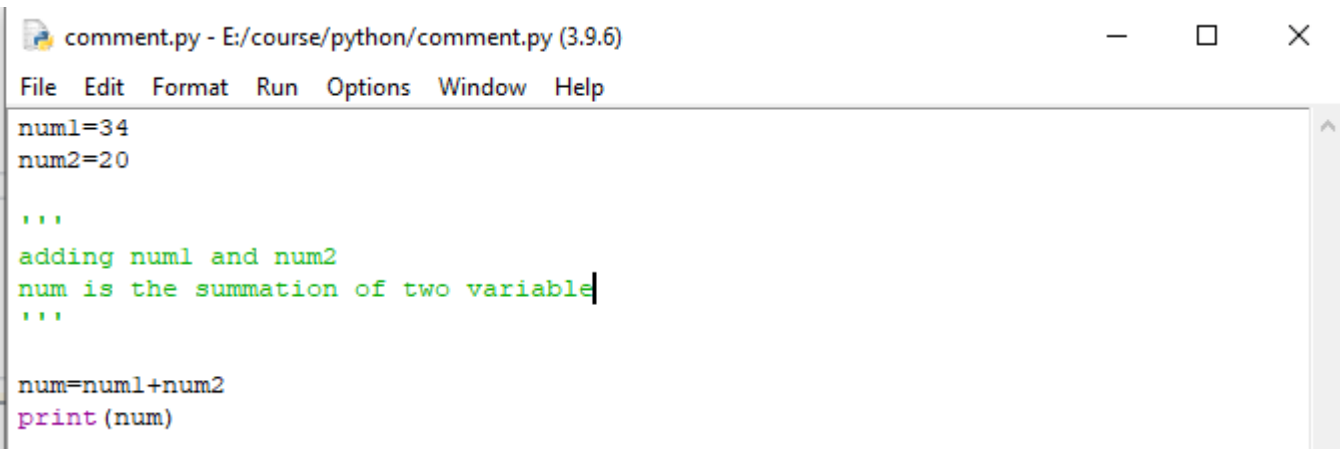
- **Comment Using #**



The screenshot shows a window titled "comment.py - E:/course/python/comment.py (3.9.6)". The menu bar includes File, Edit, Format, Run, Options, Window, and Help. The code in the editor is as follows:

```
num1=34
num2=20
num=num1+num2 #adding num1 and num2
print(num)
```

- **Comment using triple-quotation**



The screenshot shows a window titled "comment.py - E:/course/python/comment.py (3.9.6)". The menu bar includes File, Edit, Format, Run, Options, Window, and Help. The code in the editor is as follows:

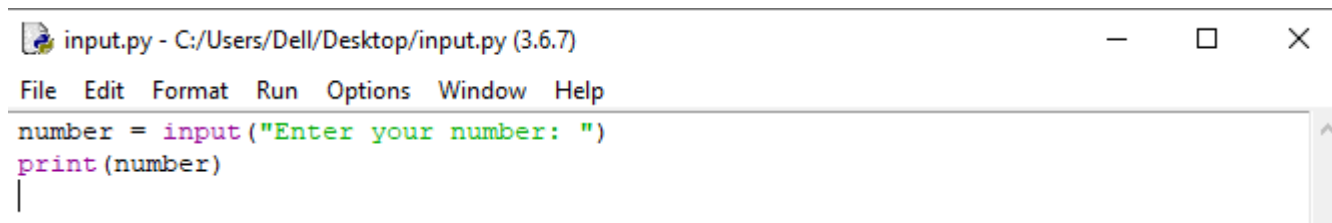
```
num1=34
num2=20

'''
adding num1 and num2
num is the summation of two variable
'''

num=num1+num2
print(num)
```

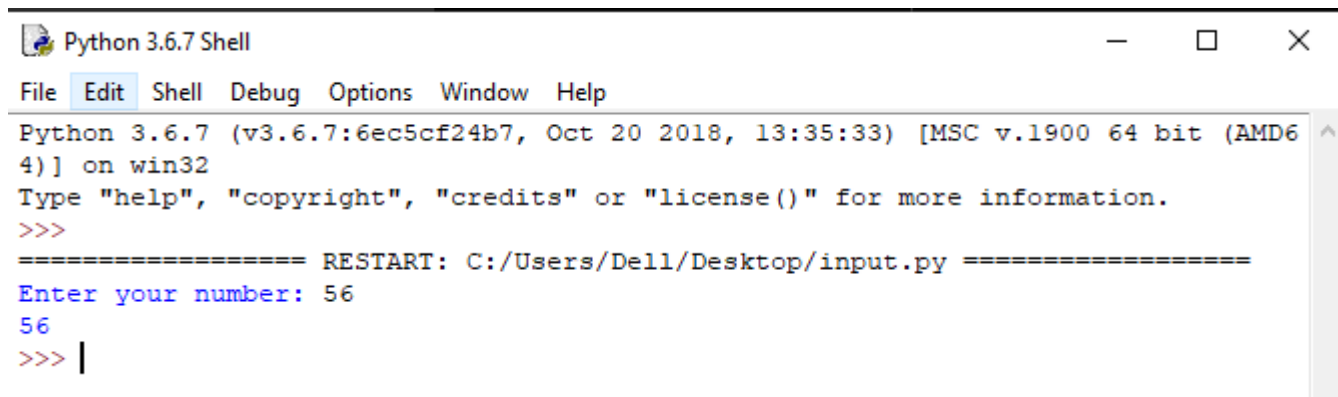
11. Input Method:

Input:



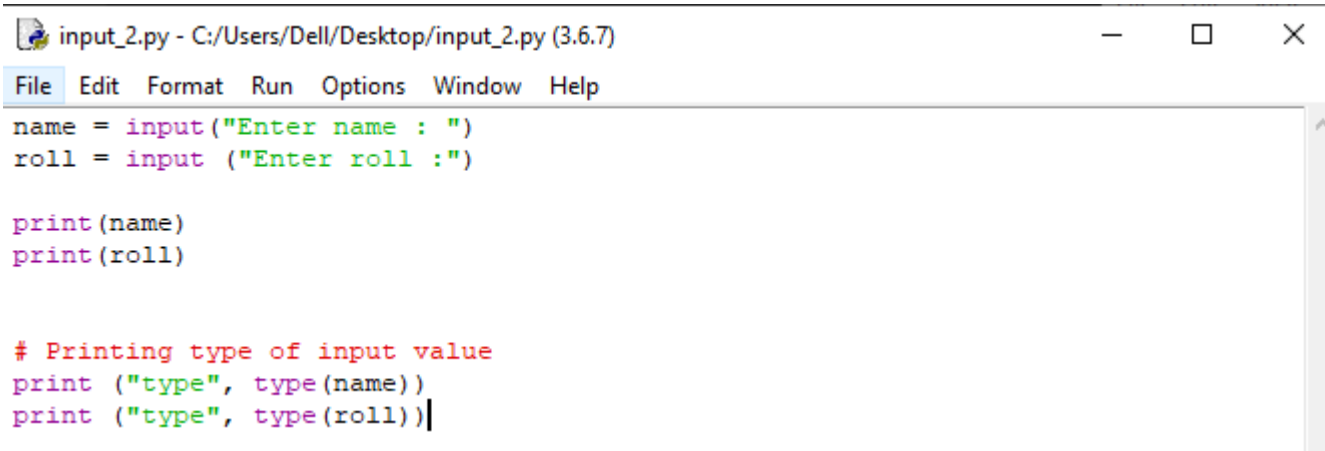
```
input.py - C:/Users/Dell/Desktop/input.py (3.6.7)
File Edit Format Run Options Window Help
number = input("Enter your number: ")
print(number)
|
```

Output:



```
Python 3.6.7 Shell
File Edit Shell Debug Options Window Help
Python 3.6.7 (v3.6.7:6ec5cf24b7, Oct 20 2018, 13:35:33) [MSC v.1900 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/Dell/Desktop/input.py =====
Enter your number: 56
56
>>> |
```

Input:

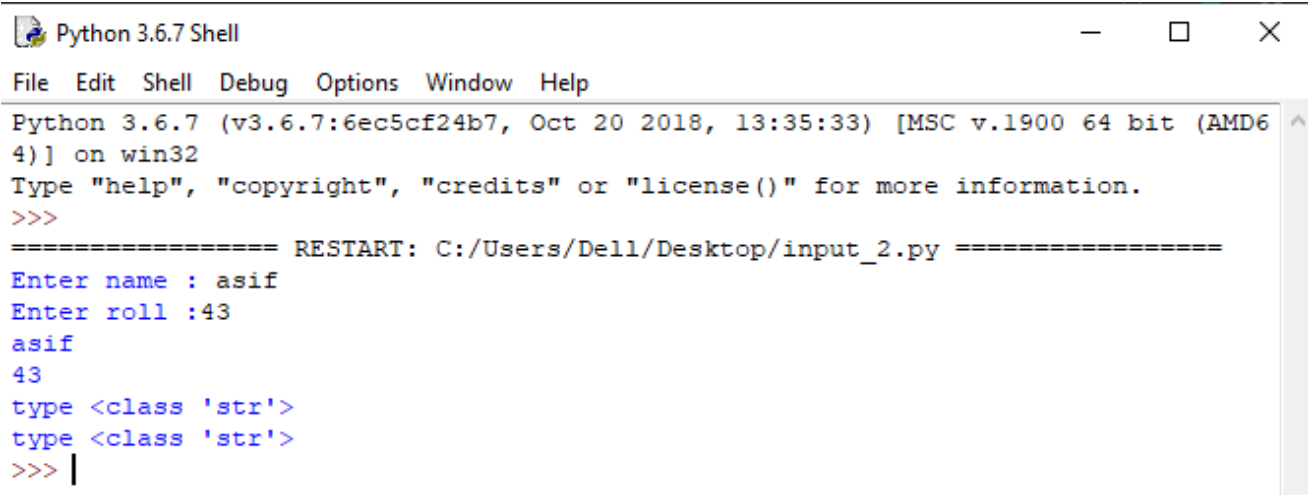


```
input_2.py - C:/Users/Dell/Desktop/input_2.py (3.6.7)
File Edit Format Run Options Window Help
name = input("Enter name : ")
roll = input("Enter roll :")

print(name)
print(roll)

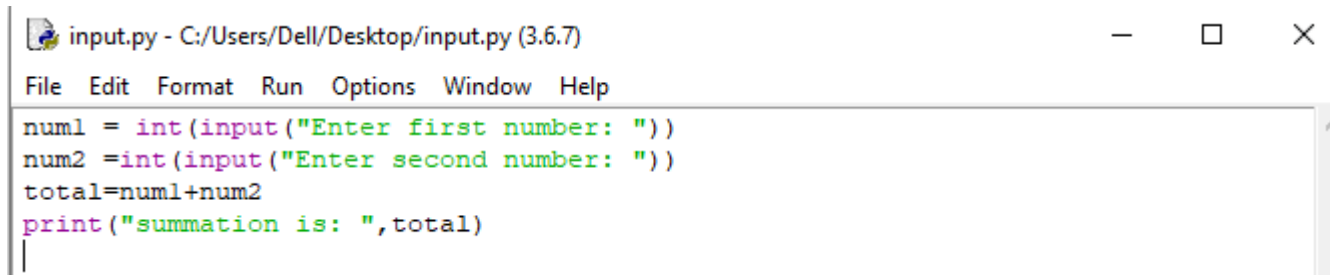
# Printing type of input value
print("type", type(name))
print("type", type(roll))
```

Output:



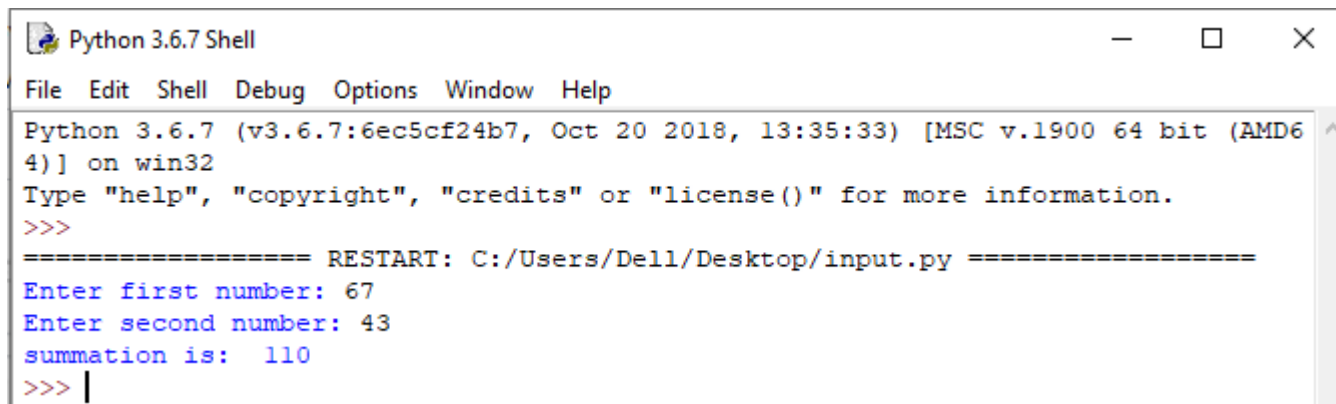
```
Python 3.6.7 Shell
File Edit Shell Debug Options Window Help
Python 3.6.7 (v3.6.7:6ec5cf24b7, Oct 20 2018, 13:35:33) [MSC v.1900 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/Dell/Desktop/input_2.py =====
Enter name : asif
Enter roll :43
asif
43
type <class 'str'>
type <class 'str'>
>>> |
```

Input:



```
input.py - C:/Users/Dell/Desktop/input.py (3.6.7)
File Edit Format Run Options Window Help
num1 = int(input("Enter first number: "))
num2 =int(input("Enter second number: "))
total=num1+num2
print("summation is: ",total)
|
```

Output:



```
Python 3.6.7 Shell
File Edit Shell Debug Options Window Help
Python 3.6.7 (v3.6.7:6ec5cf24b7, Oct 20 2018, 13:35:33) [MSC v.1900 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/Dell/Desktop/input.py =====
Enter first number: 67
Enter second number: 43
summation is: 110
>>> |
```

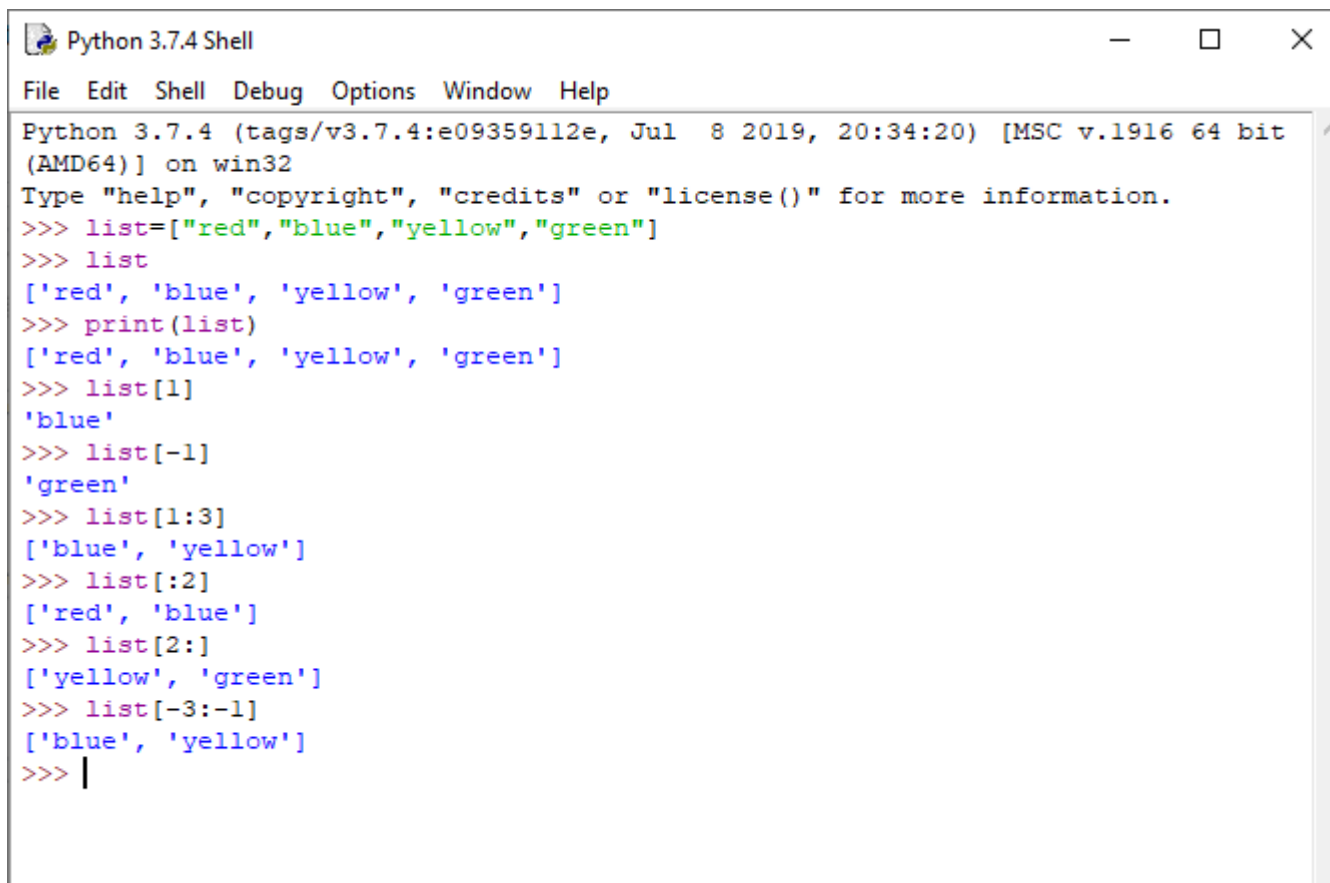
11.Python Collection:

There are 4 collection data types in the python programming language.

1. List
2. Tuple
3. Set
4. Dictionary

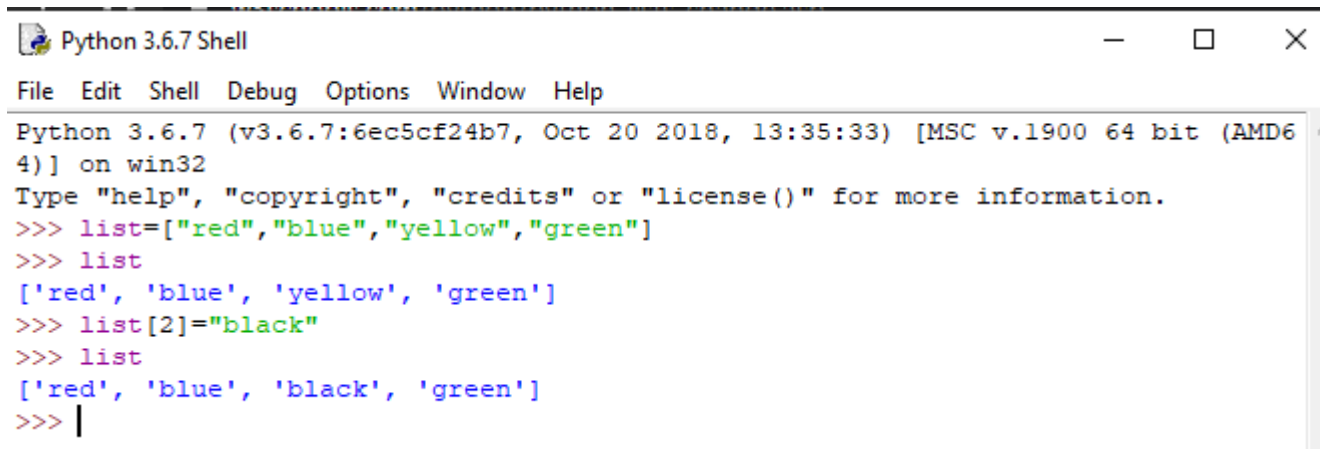
- 1.**List**:List is a collection which is ordered and changeable.
- 2.**Tuple**: Tuple is a collection which is ordered and unchangeable
- 3.**Set** :Set is a collection which is unordered and unindexed.
- 4.**Dictionary**:Dictionary is a collection which is unordered,changeable and indexed.

- **List:**



```
Python 3.7.4 Shell
File Edit Shell Debug Options Window Help
Python 3.7.4 (tags/v3.7.4:e09359112e, Jul 8 2019, 20:34:20) [MSC v.1916 64 bit
(AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> list=["red","blue","yellow","green"]
>>> list
['red', 'blue', 'yellow', 'green']
>>> print(list)
['red', 'blue', 'yellow', 'green']
>>> list[1]
'blue'
>>> list[-1]
'green'
>>> list[1:3]
['blue', 'yellow']
>>> list[:2]
['red', 'blue']
>>> list[2:]
['yellow', 'green']
>>> list[-3:-1]
['blue', 'yellow']
>>> |
```

- Change Item Value

A screenshot of a Python 3.6.7 Shell window. The window has a title bar with the text "Python 3.6.7 Shell" and standard window controls (minimize, maximize, close). Below the title bar is a menu bar with "File", "Edit", "Shell", "Debug", "Options", "Window", and "Help". The main area of the window displays the following text:

```
Python 3.6.7 (v3.6.7:6ec5cf24b7, Oct 20 2018, 13:35:33) [MSC v.1900 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> list=["red","blue","yellow","green"]
>>> list
['red', 'blue', 'yellow', 'green']
>>> list[2]="black"
>>> list
['red', 'blue', 'black', 'green']
>>> |
```

- Add and remove value from List

```
>>> color=['red','blue','yellow','green']
>>> color
['red', 'blue', 'yellow', 'green']
>>> color[2]='black'
>>> color
['red', 'blue', 'black', 'green']
>>> color.append('orange')
>>>
>>> color
['red', 'blue', 'black', 'green', 'orange']
>>> color.remove('blue')
>>> color
['red', 'black', 'green', 'orange']
>>> color.insert(2,'pink')
>>> color
['red', 'black', 'pink', 'green', 'orange']
>>>
```

- **Tuple:**

A Tuple is a collection which is ordered and unchangeable. In python tuples are written in round brackets.

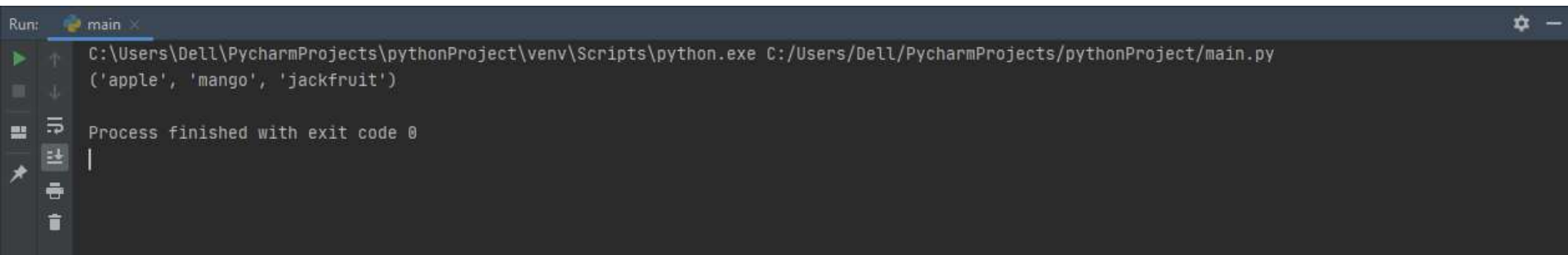
Tuples are:

1. Ordered
2. Unchangeable
3. Allow Duplicates

Input :

```
tuple=("apple","mango","jackfruit")  
print(tuple)
```

Output:

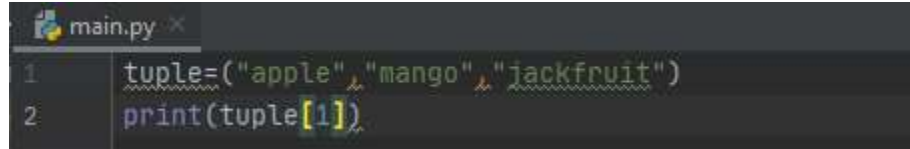


The image shows a PyCharm Run console window. The title bar says "Run: main x". The console output shows the command being executed: `C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py`. The output of the script is `('apple', 'mango', 'jackfruit')`. Below the command, it says "Process finished with exit code 0". The left sidebar contains icons for Run, Debug, and other IDE functions.

```
Run: main x  
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py  
( 'apple', 'mango', 'jackfruit')  
Process finished with exit code 0
```

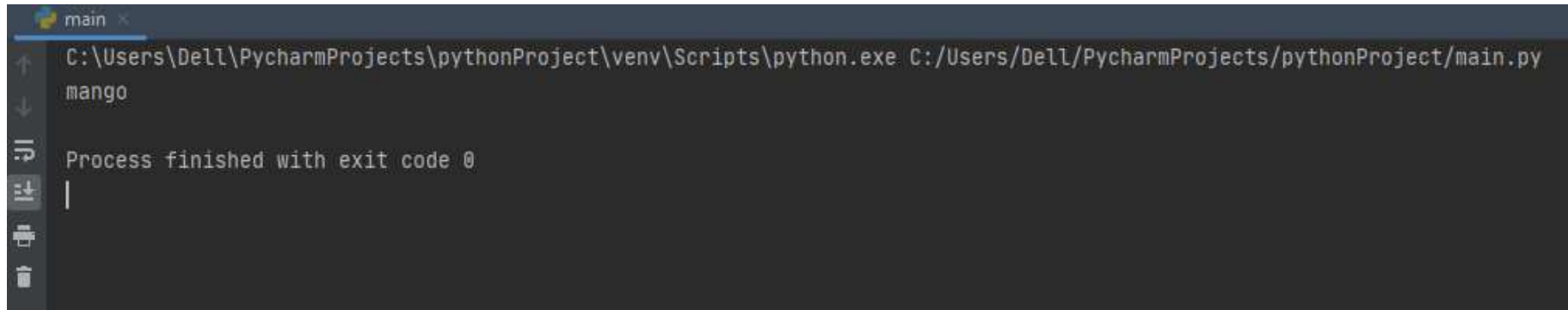
Access Tuple Items:

Input:

A screenshot of a code editor window titled 'main.py'. It contains two lines of Python code. Line 1: `tuple=("apple","mango","jackfruit")`. Line 2: `print(tuple[1])`. The code is syntax-highlighted with green for strings, blue for the variable name, and purple for the print function. The index `[1]` in the second line is highlighted with a yellow background.

```
1 tuple=("apple","mango","jackfruit")
2 print(tuple[1])
```

Output:

A screenshot of a terminal window titled 'main'. It shows the command prompt path for running the script, followed by the output 'mango' and a confirmation that the process finished with exit code 0. The terminal has a dark background with light gray text.

```
main x
C:\Users\De11\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/De11/PycharmProjects/pythonProject/main.py
mango
Process finished with exit code 0
|
```

Negative Indexing:

Negative indexing means beginning from the end, `-1` refers to the last item, `-2` refers to the second last item etc.

Input:

```
main.py x
1 tuple=("apple","mango","jackfruit")
2 print(tuple[-1])
```

Output:

```
main x
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
jackfruit
Process finished with exit code 0
```

Range of Index:

Input:

```
main.py x
1 tuple=("red" "blue" "yellow" "green" "orange" "black" "pink")
2 print(tuple[2:5])
```

Output:

```
main x
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
('yellow', 'green', 'orange')

Process finished with exit code 0
```

Range of Negative Index:

Input:

```
main.py x
1 tuple=("red","blue","yellow","green","orange","black","pink")
2 print(tuple[-4:-2])
```

Output:

```
main x
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
('green', 'orange')
Process finished with exit code 0
```

Change Tuple Value:

Input:

```
main.py x
1 x=("red","yellow","green")
2 y=list(x)
3 y[1]="blue"
4 x=tuple(y)
5 |
6 print(x)
```

Output:

```
main x
C:\Users\DeLL\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/DeLL/PycharmProjects/pythonProject/main.py
('red', 'blue', 'green')

Process finished with exit code 0
```

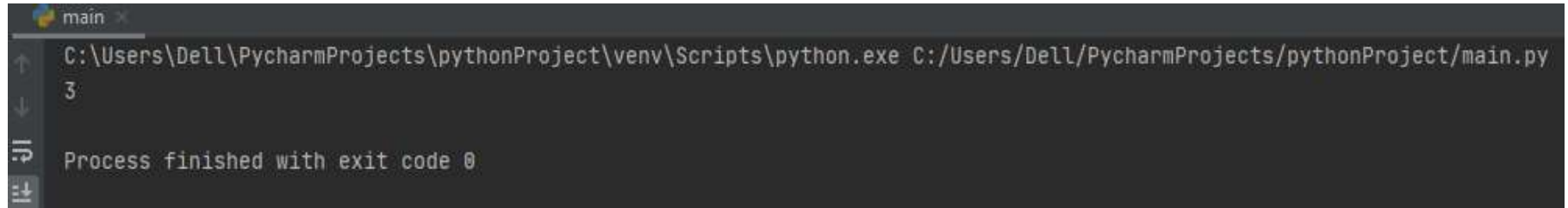
Tuple Length:

To determine how many items a tuple has, use the `len()` method:

Input:

```
1 tuple = ("apple", "mango", "jackfruit")
2 print(len(tuple))
```

Output:

A screenshot of a terminal window with a dark background. The title bar at the top shows a file icon and the text 'main x'. The terminal displays the command path 'C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py' followed by the output '3'. At the bottom, it says 'Process finished with exit code 0'. On the left side of the terminal, there is a vertical toolbar with icons for running, debugging, and other IDE functions.

```
main x
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
3
Process finished with exit code 0
```

Add Two Tuple:

Input:

```
main.py x
1 tuple1 = ("a", "b", "c")
2 tuple2 = (1, 2, 3)
3
4 tuple3 = tuple1 + tuple2
5 print(tuple3)
```

Output:

```
main x
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
('a', 'b', 'c', 1, 2, 3)
Process finished with exit code 0
```

- **Set:**

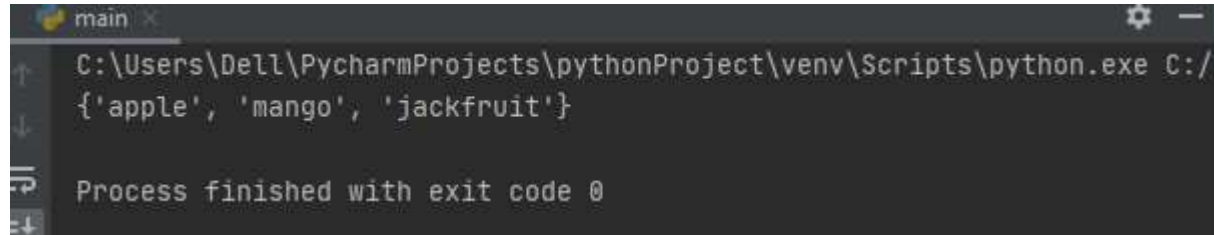
A set is a collection which is unordered and unindexed. In Python, sets are written with curly brackets

Input:

A screenshot of a code editor window titled 'main.py'. It contains two lines of Python code: line 1 is 'set = {"apple", "mango", "jackfruit"}' and line 2 is 'print(set)'.

```
1 set = {"apple", "mango", "jackfruit"}
2 print(set)
```

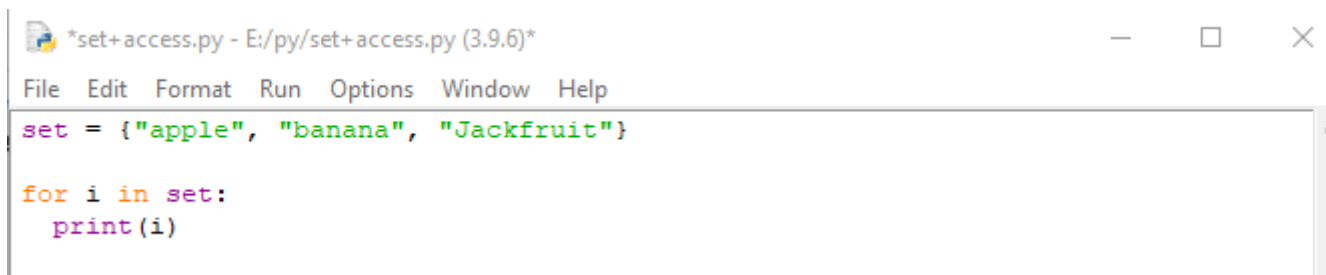
Output:

A screenshot of a terminal window titled 'main'. It shows the command prompt path 'C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/', followed by the output of the script: {'apple', 'mango', 'jackfruit'}. At the bottom, it says 'Process finished with exit code 0'.

```
main
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/
{'apple', 'mango', 'jackfruit'}
Process finished with exit code 0
```

Access Set Items:

Input:

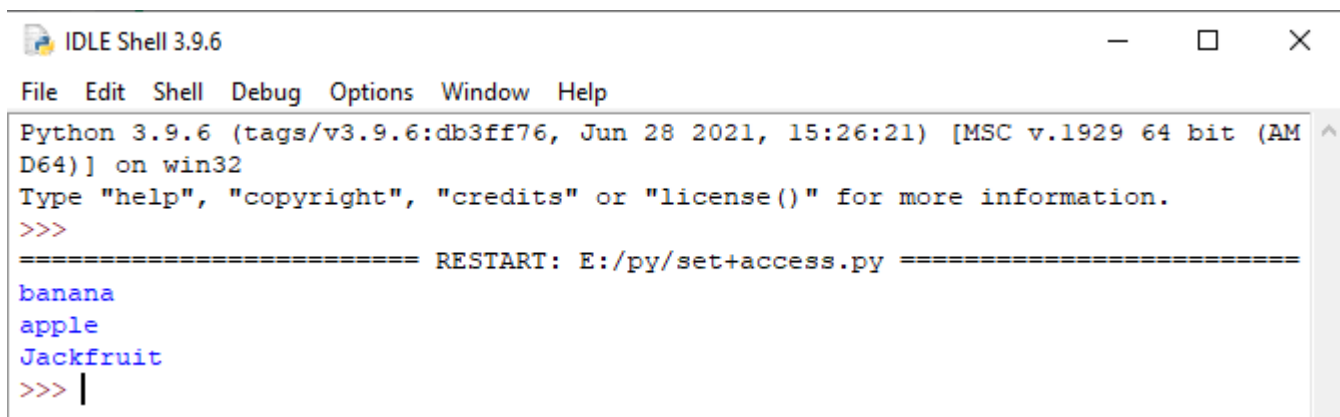


The screenshot shows a Python IDE window titled '*set+access.py - E:/py/set+access.py (3.9.6)*'. The menu bar includes File, Edit, Format, Run, Options, Window, and Help. The code in the editor is as follows:

```
set = {"apple", "banana", "Jackfruit"}

for i in set:
    print(i)
```

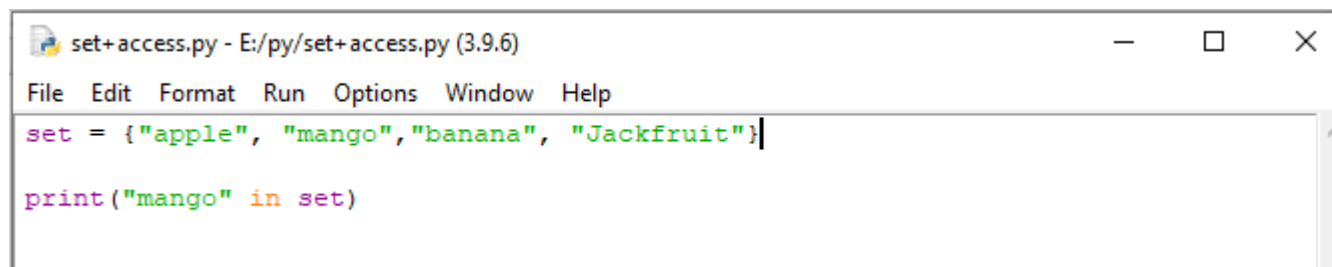
Output:



The screenshot shows the IDLE Shell 3.9.6 window. The menu bar includes File, Edit, Shell, Debug, Options, Window, and Help. The shell displays the following text:

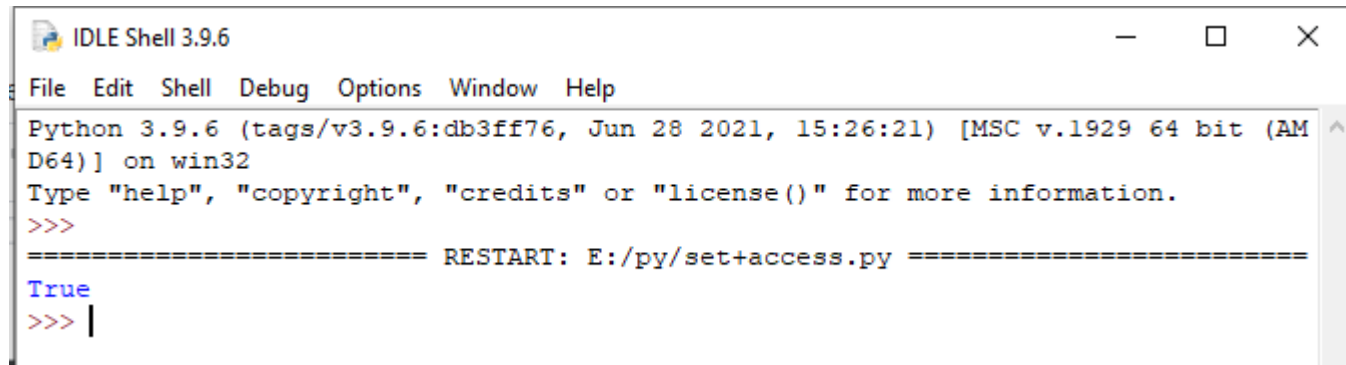
```
Python 3.9.6 (tags/v3.9.6:db3ff76, Jun 28 2021, 15:26:21) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: E:/py/set+access.py =====
banana
apple
Jackfruit
>>> |
```

Input:



```
set+access.py - E:/py/set+access.py (3.9.6)
File Edit Format Run Options Window Help
set = {"apple", "mango", "banana", "Jackfruit"}
print("mango" in set)
```

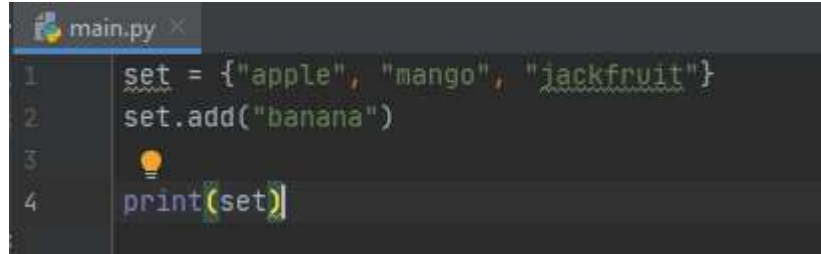
Output:



```
IDLE Shell 3.9.6
File Edit Shell Debug Options Window Help
Python 3.9.6 (tags/v3.9.6:db3ff76, Jun 28 2021, 15:26:21) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: E:/py/set+access.py =====
True
>>> |
```

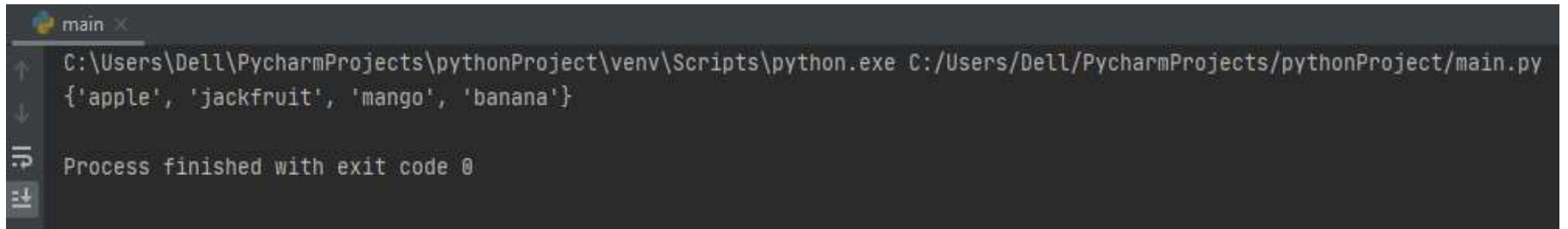
Add Items:

Input:



```
main.py x
1  set = {"apple", "mango", "jackfruit"}
2  set.add("banana")
3
4  print(set)
```

Output:



```
main x
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
{'apple', 'jackfruit', 'mango', 'banana'}

Process finished with exit code 0
```

Add Multiple Items:

Input:

```
main.py x
1  set = {"apple", "mango", "jackfruit"}
2  set.update(["banana", "pineapple", "watermelon", "orange"])
3
4  print(set)
```

Output:

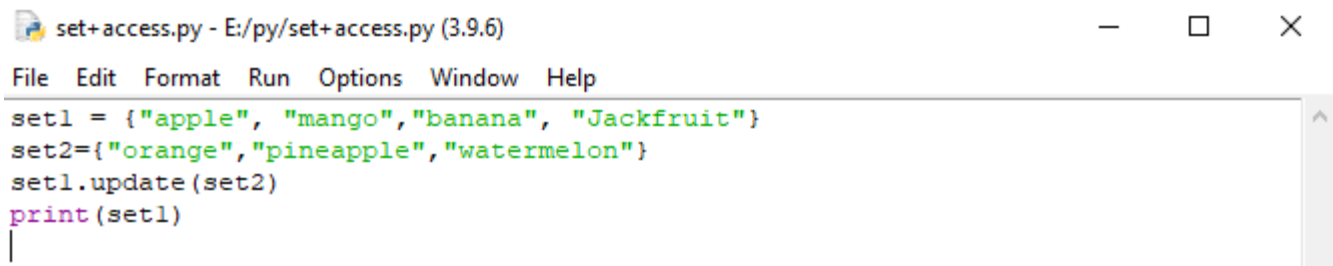
```
main x
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.p
{'mango', 'orange', 'apple', 'banana', 'jackfruit', 'watermelon', 'pineapple'}

Process finished with exit code 0
```

Add Sets:

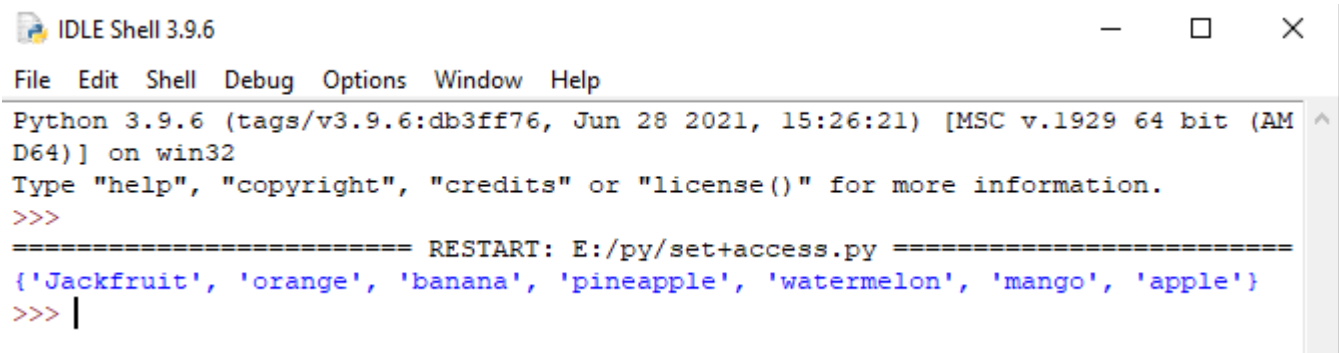
The `update ()` method inserts the items in set2 to set1.

Input:



```
File Edit Format Run Options Window Help
set1 = {"apple", "mango", "banana", "Jackfruit"}
set2={"orange", "pineapple", "watermelon"}
set1.update(set2)
print(set1)
|
```

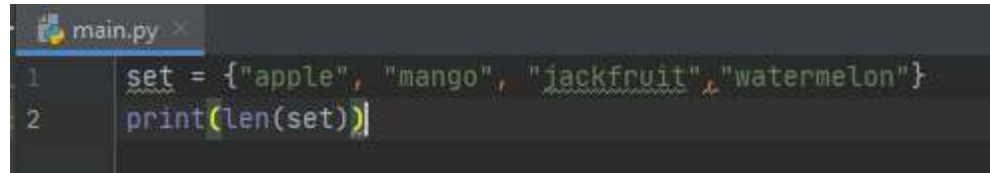
Output:



```
IDLE Shell 3.9.6
File Edit Shell Debug Options Window Help
Python 3.9.6 (tags/v3.9.6:db3ff76, Jun 28 2021, 15:26:21) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: E:/py/set+access.py =====
{'Jackfruit', 'orange', 'banana', 'pineapple', 'watermelon', 'mango', 'apple'}
>>> |
```

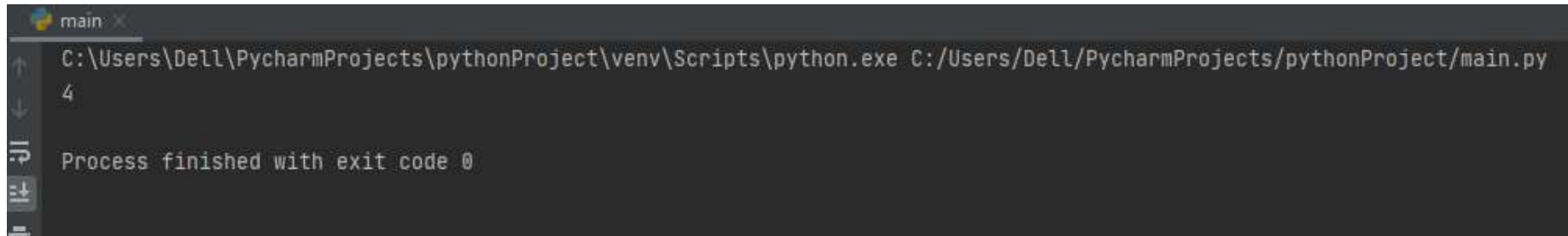
Length Of a Set:

Input:



```
main.py x
1  set = {"apple", "mango", "jackfruit", "watermelon"}
2  print(len(set))
```

Output:

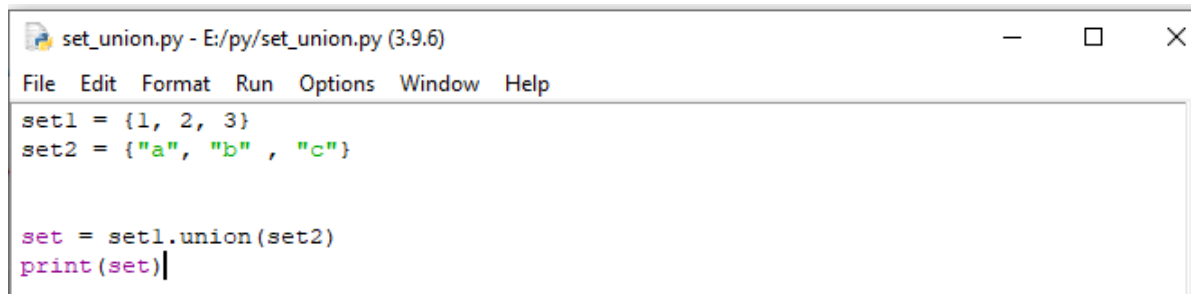


```
main x
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
4
Process finished with exit code 0
```

Join To Sets:

The `union ()` method returns a new set with all items from both sets:

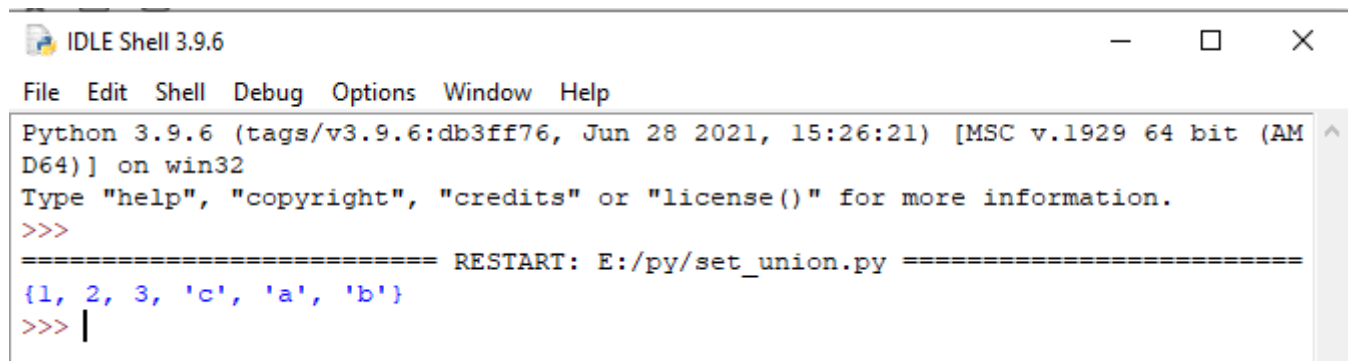
Input:



```
set_union.py - E:/py/set_union.py (3.9.6)
File Edit Format Run Options Window Help
set1 = {1, 2, 3}
set2 = {"a", "b", "c"}

set = set1.union(set2)
print(set)
```

Output:



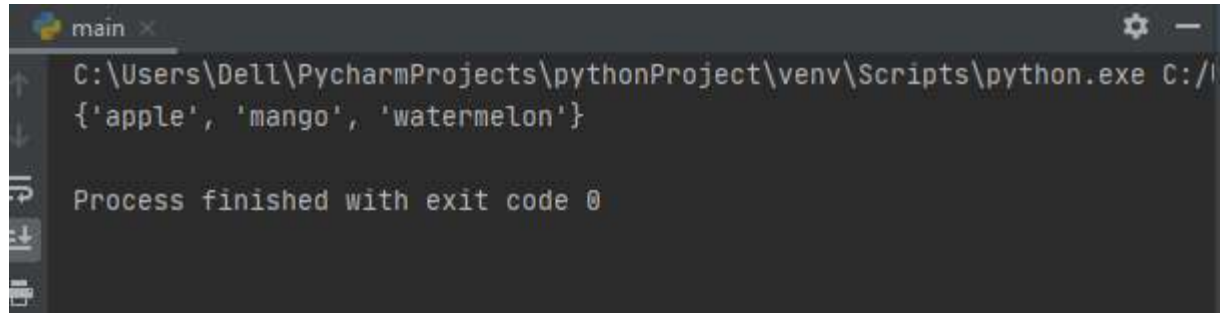
```
IDLE Shell 3.9.6
File Edit Shell Debug Options Window Help
Python 3.9.6 (tags/v3.9.6:db3ff76, Jun 28 2021, 15:26:21) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: E:/py/set_union.py =====
{1, 2, 3, 'c', 'a', 'b'}
>>> |
```

Remove an Item:

Input:

```
1  set = {"apple", "mango", "jackfruit", "watermelon"}
2  set.remove("jackfruit")
3
4  print(set)
```

Output:



```
main x
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/l
{'apple', 'mango', 'watermelon'}

Process finished with exit code 0
```

- **Python Dictionary:**

A dictionary is a collection which is unordered, changeable and indexed. In Python dictionaries are written with curly brackets, and they have keys and values.

Example:

Input:

```
1 dict = {  
2     "name": "Hridoy",  
3     "class": "Nine",  
4     "roll": 14  
5 }  
6 print(dict)
```

Output:

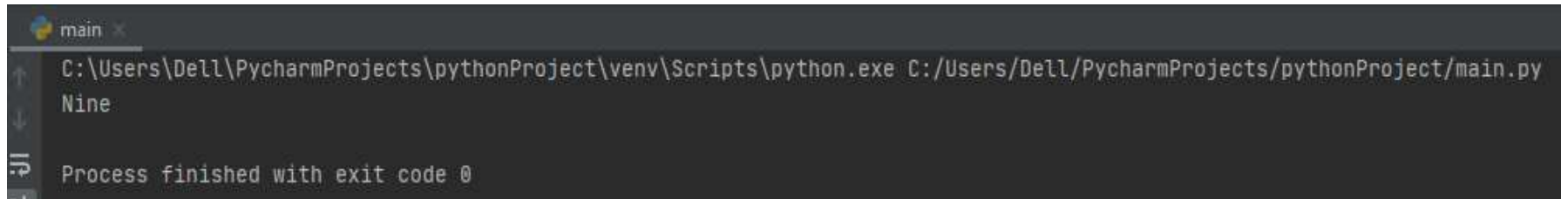
```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C  
{'name': 'Hridoy', 'class': 'Nine', 'roll': 14}  
  
Process finished with exit code 0
```

Accessing Items:

Input:

```
1 dict = {  
2     "name": "Hridoy",  
3     "class": "Nine",  
4     "roll": 14  
5 }  
6 x=dict["class"]  
7 print(x)
```

Output:



The screenshot shows a terminal window with a dark background. At the top, there is a tab labeled 'main'. Below the tab, the command prompt shows the full path to the Python interpreter and the script: `C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py`. The output of the script is displayed on the next line as `Nine`. At the bottom, a status message reads `Process finished with exit code 0`.

Change Value:

Input:

```
home > asif > Course > PPT > python > Untitled-1.py > ...
1 dict={
2     'Name': 'Hridoy',
3     'Class': 'Nine',
4     'Roll': 8
5 }
6 dict['Roll']=12
7
8 print(dict)
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL
[Running] python -u "/home/asif/Course/PPT/python/Untitled-1.py"
{'Name': 'Hridoy', 'Class': 'Nine', 'Roll': 12}

[Done] exited with code=0 in 0.04 seconds
```

Dictionary Length:

Input:

```
1 dict={
2     "name": "Hridoy",
3     "class": "Nine",
4     "roll": 14
5 }
6 print(len(dict))
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
3

Process finished with exit code 0
```

Adding items:

Input:

```
1 dict={
2     'Name': "Asif",
3     'Class':7,
4     'Roll': 5
5 }
6 dict['Section']='Science'
7 print(dict)
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL
[Running] python -u "/home/asif/Course/PPT/python/Untitled-1.py"
{'Name': 'Asif', 'Class': 7, 'Roll': 5, 'Section': 'Science'}

[Done] exited with code=0 in 0.052 seconds
```

Removing Items:

Input:

```
1 dict={  
2     "name": "Hridoy",  
3     "class": "Nine",  
4     "roll": 14  
5 }  
6 del dict["roll"]  
7 print(dict)
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\pyt  
{'name': 'Hridoy', 'class': 'Nine'}  
  
Process finished with exit code 0
```

Copy a Dictionary:

Input:

```
1 dict={
2     "name": "Hridoy",
3     "class": "Nine",
4     "roll": 14
5 }
6 copy_dict=dict.copy()
7 print(copy_dict)
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Script
{'name': 'Hridoy', 'class': 'Nine', 'roll': 14}

Process finished with exit code 0
```

Nested Dictionaries:

Input:

```
1 student_list={
2     "student_1":{
3         "name": "Hridoy",
4         "class": "Seven",
5         "roll": 14
6     },
7     "student_2":{
8         "name": "Arafat",
9         "class": "Seven",
10        "roll": 34
11    },
12    "student_3":{
13        "name": "Osman",
14        "class": "Seven",
15        "roll": 7
16    }
17 }
18 print(student_list)
```

Output:

```
main x
rs\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
ent_1': {'name': 'Hridoy', 'class': 'Seven', 'roll': 14}, 'student_2': {'name': 'Arafat', 'class': 'Seven', 'roll': 34}, 'student_3':

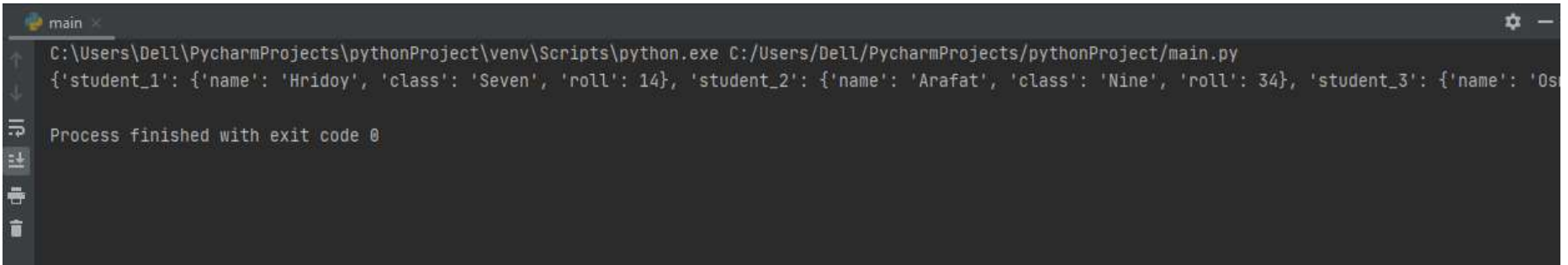
s finished with exit code 0
```

```
'student_3': {'name': 'Osman', 'class': 'Seven', 'roll': 7}}
```

Input:

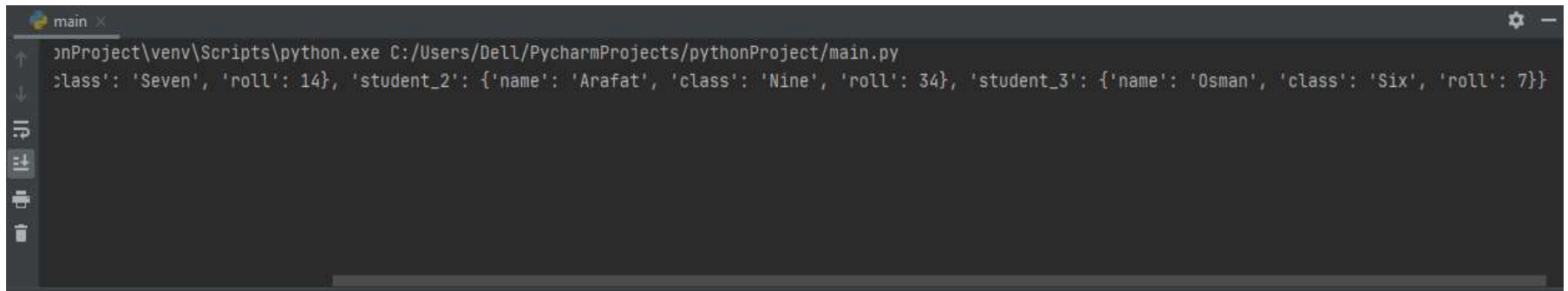
```
1  student_1= {  
2      "name": "Hridoy",  
3      "class": "Seven",  
4      "roll": 14  
5  }  
6  student_2= {  
7      "name": "Arafat",  
8      "class": "Nine",  
9      "roll": 34  
10 }  
11 student_3= {  
12     "name": "Osman",  
13     "class": "Six",  
14     "roll": 7  
15 }  
16 student_list = {  
17     "student_1": student_1,  
18     "student_2": student_2,  
19     "student_3": student_3  
20 }  
21 print(student_list)
```

Output:



The image shows a PyCharm console window with a dark theme. The title bar at the top says "main" with a close button. The console output consists of two lines: the first line shows the command to run a Python script, and the second line shows the successful completion of the script.

```
main x [gear icon] [minus icon]
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
{'student_1': {'name': 'Hridoy', 'class': 'Seven', 'roll': 14}, 'student_2': {'name': 'Arafat', 'class': 'Nine', 'roll': 34}, 'student_3': {'name': 'Osman', 'class': 'Six', 'roll': 7}}
Process finished with exit code 0
```



The image shows a PyCharm console window with a dark theme. The title bar at the top says "main" with a close button. The console output shows a command to run a Python script, but the output is cut off, showing only the first part of the dictionary.

```
main x [gear icon] [minus icon]
venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
{'class': 'Seven', 'roll': 14}, 'student_2': {'name': 'Arafat', 'class': 'Nine', 'roll': 34}, 'student_3': {'name': 'Osman', 'class': 'Six', 'roll': 7}}
```

12. Python If..Else:

Python condition and if statement:

- Equal: $a==b$
- Not Equal: $a!=b$
- Greater than: $a>b$
- Greater or equal than: $a>=b$
- Less than: $a<b$
- Less or equal than: $a<=b$

Example:

If statement:

```
1 a = 23
2 b = 9
3 if a > b:
4     print("a is greater than b")
```

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
a is greater than b
```

```
Process finished with exit code 0
```

Else:

Input:

```
1 a = 399
2 b = 206
3 if a > b:
4     print("a is greater than b")
5 else:
6     print("a is not greater than b")
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
a is greater than b

Process finished with exit code 0
```

Elif:

Input:

```
1 a = 200
2 b = 300
3 if a > b:
4     print("a is greater than b")
5 elif a == b:
6     print("a and b are equal")
7 else:
8     print("b is greater than a")
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
b is greater than a

Process finished with exit code 0
```

ShortHand If:

Input:

```
1  a=76
2  b=39
3  if a > b: print("a is greater than b")
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
a is greater than b

Process finished with exit code 0
```

ShortHand if..Else:

Input:

```
1 a = 322
2 b = 233
3 print("a is greater than b") if a > b else print("a is not greater than b")
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
a is greater than b
```

```
Process finished with exit code 0
```

And:

Input:

```
1 a = 1003
2 b = 4076
3 c = 520
4 if b > a and a > c:
5     print("Both conditions are True")
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
Both conditions are True

Process finished with exit code 0
```

Or:

Input:

```
1 a = 300
2 b = 200
3 c = 500
4 if a > b or a > c:
5     print("At least one of the conditions is True")
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
At least one of the conditions is True

Process finished with exit code 0
```

Nested If:

Input:

```
1 mark = 76
2 if mark < 35:
3     print("Grade is F")
4 if mark >= 35 and mark < 40:
5     print("Grade is D")
6 if mark >= 40 and mark < 50:
7     print("Grade is C")
8 if mark >= 50 and mark < 60:
9     print("Grade is B")
10 if mark >= 60 and mark < 70:
11     print("Grade is A-")
12 if mark >= 70 and mark < 80:
13     print("Grade is A")
14 if mark >= 80:
15     print("Grade is A+")
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
Grade is A

Process finished with exit code 0
```

Nested if using Input Method: Input:

```
1 mark =int(input("mark?"))
2 if mark<35:
3     print("Grade is F")
4 if mark >= 35 and mark <40:
5     print("Grade is D")
6 if mark >= 40 and mark <50 :
7     print("Grade is C")
8 if mark >= 50 and mark <60:
9     print("Grade is B")
10 if mark >= 60 and mark <70:
11     print("Grade is A-")
12 if mark >= 70 and mark <80:
13     print("Grade is A")
14 if mark >= 80:
15     print("Grade is A+")
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
mark?56
Grade is B

Process finished with exit code 0
```

13.Python Loop:

- While Loop
- For Loop

While Loop:

Input:

```
1 i = 1
2 while i < 5:
3     print(i)
4     i += 1
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
1
2
3
4

Process finished with exit code 0
```

Else In While Loop:

Input:

```
1 i = 1
2 while i < 5:
3     print(i)
4     i += 1
5 else:
6     print("i is no longer less than 5")
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
```

```
1
2
3
4
```

```
i is no longer less than 5
```

```
Process finished with exit code 0
```

- **For Loop:**

Input:

```
1 fruits = ["apple", "mango", "jackfruit"]
2 for x in fruits:
3     print(x)
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
apple
mango
jackfruit
```

```
Process finished with exit code 0
```

Looping Through a String

Input:

```
1 for x in "mango":  
2     print(x)
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py  
m  
a  
n  
g  
o  
  
Process finished with exit code 0
```

Else in For Loop

Input:

```
1  for x in range(6):  
2      print(x)  
3  else:  
4      print("Finished!")
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py  
0  
1  
2  
3  
4  
5  
Finished!
```

Nested Loops

Input:

 nested.py - D:\asif-office-work\python_code\nested.py (3.10.2)

File Edit Format Run Options Window Help

```
shade = ["light", "deep"]
colour = ["red", "blue", "green"]

for x in shade:
    for y in colour:
        print(x, y)
```

Output:

Python 3.10.2 (tags/v3.10.2:a58ebcc, Jan 17 2022, 14:12:15) [MSC v.1929 64 bit (AMD64)] on win32

Type "help", "copyright", "credits" or "license()" for more information.

===== RESTART: D:\asif-office-work\python_code\nested.py =====

```
light red
light blue
light green
deep red
deep blue
deep green
```

14. Python Statement:

Pass Statement:

Input:

```
1 a = 499
2 b = 200
3
4 if b > a:
5     pass
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
```

```
Process finished with exit code 0
```

- **pass Statement in For loop**

for loops cannot be empty, but if you for some reason have a **for** loop with no content, put in the **pass** statement to avoid getting an error.

```
1 for x in [0, 1, 2, 3, 4]:  
2     pass
```

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
```

```
Process finished with exit code 0
```

- The continue Statement

With the `continue` statement we can stop the current iteration of the loop, and continue with the next:

Example:

Do not print `jackfruit` :

Input:

```
1 fruits = ["apple", "mango", "jackfruit", "orange"]
2 for x in fruits:
3     if x == "jackfruit":
4         continue
5     print(x)
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
apple
mango
orange

Process finished with exit code 0
```

Continue Statement:

Input:

```
1 i = 0
2 while i < 5:
3     i += 1
4     if i == 3:
5         continue
6     print(i)
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
```

```
1
2
4
5
```

```
Process finished with exit code 0
```

- **Break Statement:**

Input:

```
1 i = 1
2 while i < 5:
3     print(i)
4     if i == 3:
5         break
6     i += 1
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
1
2
3

Process finished with exit code 0
```

break Statement in For Loop:

Input:

```
1 fruits = ["apple", "mango", "jackfruit", "orange"]
2 for x in fruits:
3     print(x)
4     if x == "jackfruit":
5         break
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
apple
mango
jackfruit

Process finished with exit code 0
```

Exit the loop when `x` is "banana", but this time the break comes before the print:

Input:

```
1  fruits = ["apple", "mango", "jackfruit", "orange"]
2  for x in fruits:
3      if x == "jackfruit":
4          break
5      print(x)
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
apple
mango

Process finished with exit code 0
```

15. The range() Function

Input:

```
1 for x in range(5):  
2     print(x)
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py  
0  
1  
2  
3  
4  
  
Process finished with exit code 0
```

Example:

Input:

```
1 for x in range(2, 7):  
2     print(x)
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py  
2  
3  
4  
5  
6  
  
Process finished with exit code 0
```

Example

Input:

```
1 for x in range(5, 50, 15):  
2     print(x)
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py  
5  
20  
35  
  
Process finished with exit code 0
```

16.Function:

Creating a function:

```
1 def my_function():  
2     print("Hello world!")
```

Calling a function:

```
1 def my_function():  
2     print("Hello world!")  
3  
4     my_function()
```

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py  
Hello world!
```

```
Process finished with exit code 0
```

❑ Parameters or Arguments?

The terms *parameter* and *argument* can be used for the same thing: information that are passed into a function.

From a function's perspective:

A parameter is the variable listed inside the parentheses in the function definition.

An argument is the value that is sent to the function when it is called.

Input:

```
def my_function(fname):  
    print("Hello "+fname)  
  
my_function("asif")
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py  
Hello asif  
  
Process finished with exit code 0
```

Input:

```
def my_function(city = "Dhaka"):
    print("I am from " + city)

my_function()
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
I am from Dhaka

Process finished with exit code 0
```

Input:

```
def my_function(fname, lname):  
    print(fname+" "+lname)  
  
my_function("Asif", "Mredha")
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py  
Asif Mredha
```

```
Process finished with exit code 0
```

Arbitrary Arguments, *args

If you do not know how many arguments that will be passed into your function, add a `*` before the parameter name in the function definition.

This way the function will receive a *tuple* of arguments, and can access the items accordingly:

Input:

```
def my_function(*city):  
    print("I live in " + city[1])  
  
my_function("Rajshahi", "Dhaka", "Sylhet")
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py  
I live in Dhaka  
  
Process finished with exit code 0
```

Keyword Arguments

You can also send arguments with the *key = value* syntax.

This way the order of the arguments does not matter.

Input:

```
def my_function(city1, city2, city3):  
    print("Most populated city in bangladesh is " + city3)  
  
my_function(city1 = "chittagong", city2 = "khulna", city3 = "Dhaka")
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py  
Most populated city in bangladesh is Dhaka  
  
Process finished with exit code 0
```

Arbitrary Keyword Arguments, **kwargs

If you do not know how many keyword arguments that will be passed into your function, add two asterisk: `**` before the parameter name in the function definition.

This way the function will receive a *dictionary* of arguments, and can access the items accordingly:

Input:

```
def my_function(**lname):  
    print("His last name is " + lname["lastname"])  
  
my_function(firstname = "Asif", lastname = "Mredha")
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py  
His last name is Mredha  
  
Process finished with exit code 0
```

Passing a List as an Argument

Input:

```
def my_function(fruits):  
    for x in fruits:  
        print(x)  
  
fruits = ["apple", "banana", "cherry"]  
  
my_function(fruits)
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  
  
Try the new cross-platform PowerShell https://aka.ms/pscore6  
  
PS C:\Users\Microland IT> & "C:/Users/Microland IT/AppData/Local/  
apple  
banana  
cherry
```

Return Values

To let a function return a value, use the `return` statement:

Input:

```
def my_function(x):  
    return 5 * x  
  
print(my_function(3))  
print(my_function(5))  
print(my_function(12))
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py  
15  
25  
60  
  
Process finished with exit code 0
```

17. Python Lambda:

Input:

```
add=lambda x: x+5  
print(add(3))
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py  
8  
  
Process finished with exit code 0
```

Input:

```
a = lambda x, y : x ** y  
print(a(5, 3))
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py  
125  
|  
Process finished with exit code 0
```

Input:

```
def my_function(n):  
    return lambda x : x * n  
  
my_doubler = my_function(2)  
  
print(my_doubler(10))
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py  
20
```

```
Process finished with exit code 0
```

Input:

```
def my_function(n):  
    return lambda x : x * n  
  
my_doubler = my_function(2)  
my_tripler = my_function(3)  
  
print(my_doubler(12))  
print(my_tripler(10))
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py  
24  
30  
  
Process finished with exit code 0
```

18.Array:

Input:

```
fruits = ["apple", "mango", "jackfruit"]
```

```
fruits = ["apple", "mango", "jackfruit"]  
print(fruits)
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py  
['apple', 'mango', 'jackfruit']  
  
Process finished with exit code 0
```

Input:

```
fruit1="apple"  
fruit2="mango"  
fruit3="jackfruit"  
  
fruits=[fruit1,fruit2,fruit3]  
print(fruits)
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py  
['apple', 'mango', 'jackfruit']  
  
Process finished with exit code 0
```

Access the Elements of an Array

Input:

```
fruits=["apple","mango","jackfruit"]  
print(fruits[2])
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py  
jackfruit
```

```
Process finished with exit code 0
```

Modify the value of array item:

Input:

```
fruits=["apple","mango","jackfruit"]  
fruits[2]="banana"  
print(fruits)
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py  
['apple', 'mango', 'banana']  
  
Process finished with exit code 0
```

Length of an array:

Input:

```
fruits=["apple","mango","jackfruit"]  
print(len(fruits))
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py  
3
```

```
Process finished with exit code 0
```

Looping Array Elements

Input:

```
fruits=["apple","mango","jackfruit"]  
for x in fruits:  
    print(x)
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py  
apple  
mango  
jackfruit  
  
Process finished with exit code 0
```

Adding Array Elements

You can use the `append()` method to add an element to an array.

Input:

```
fruits=["apple","mango","jackfruit"]  
fruits.append("orange")  
  
print(fruits)
```

Output:

```
C:\Users\DeLL\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/DeLL/PycharmProjects/pythonProject/main.py  
['apple', 'mango', 'jackfruit', 'orange']  
  
Process finished with exit code 0
```

Removing Array Elements

You can use the `pop()` method to remove an element from the array.

Input:

```
fruits=["apple","mango","jackfruit","watermelon"]  
fruits.pop(2)  
  
print(fruits)
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py  
['apple', 'mango', 'watermelon']  
  
Process finished with exit code 0
```

You can also use the `remove()` method to remove an element from the array.

Input:

```
fruits=["apple","mango","jackfruit","watermelon"]  
fruits.remove("mango")  
  
print(fruits)
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py  
['apple', 'jackfruit', 'watermelon']  
  
Process finished with exit code 0
```

19. Python math:

Python has a set of built-in math functions, including an extensive math module, that allows you to perform mathematical tasks on numbers.

Built-in Math Functions:

The `min()` and `max()` functions can be used to find the lowest or highest value in an iterable:

Input:

```
x = max(5, 10, 25)
y = min(5, 10, 25)

print("max number is ",x)
print("min number is ",y)
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
max number is  25
min number is  5

Process finished with exit code 0
```

The `abs()` function returns the absolute (positive) value of the specified number:

Input:

```
x = abs(-13.62)
print(x)
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
13.62

Process finished with exit code 0
```

The `pow(x, y)` function returns the value of x to the power of y (x^y).

Input:

```
x = pow(5, 3)
print(x)
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
125

Process finished with exit code 0
```

The Math Module

Python has also a built-in module called `math`, which extends the list of mathematical functions.

To use it, you must import the `math` module:

```
import math
```

When you have imported the `math` module, you can start using methods and constants of the module.

The `math.sqrt()` method for example, returns the square root of a number:

```
import math
x = math.sqrt(49)
print(x)
```

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
7.0
```

```
Process finished with exit code 0
```

The `math.ceil()` method rounds a number upwards to its nearest integer, and the `math.floor()` method rounds a number downwards to its nearest integer, and returns the result:

```
import math

x = math.ceil(2.6)
y = math.floor(2.6)

print(x) # returns 3
print(y) # returns 2
```

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
3
2

Process finished with exit code 0
```

The `math.pi` constant, returns the value of PI (3.14...):

Input:

```
import math
x = math.pi
print(x)
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
3.141592653589793
```

```
Process finished with exit code 0
```

20.Python Classes/Objects

Python is an object oriented programming language.

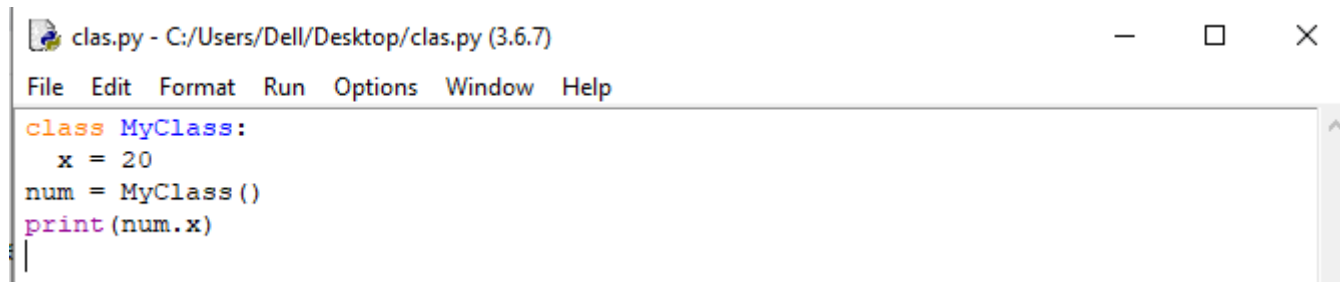
Almost everything in Python is an object, with its properties and methods.

Create a Class

To create a class, use the keyword **class**:

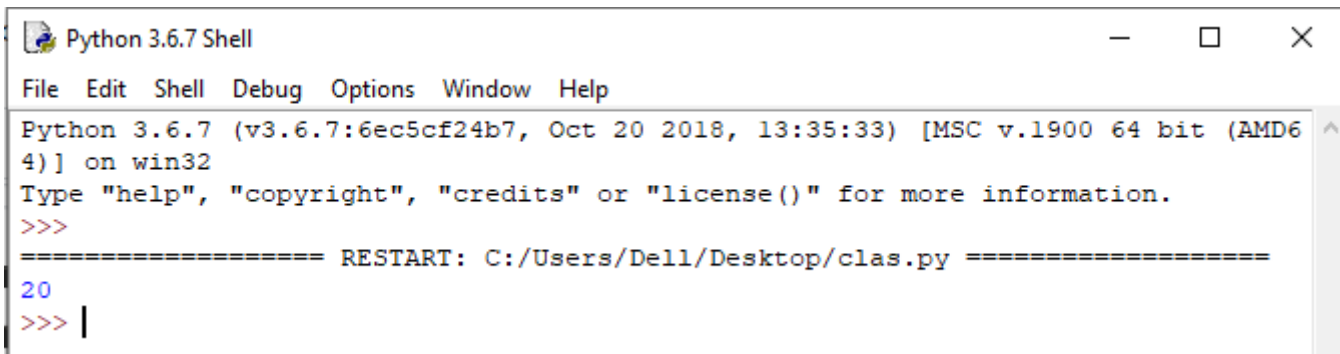
Example

Input:



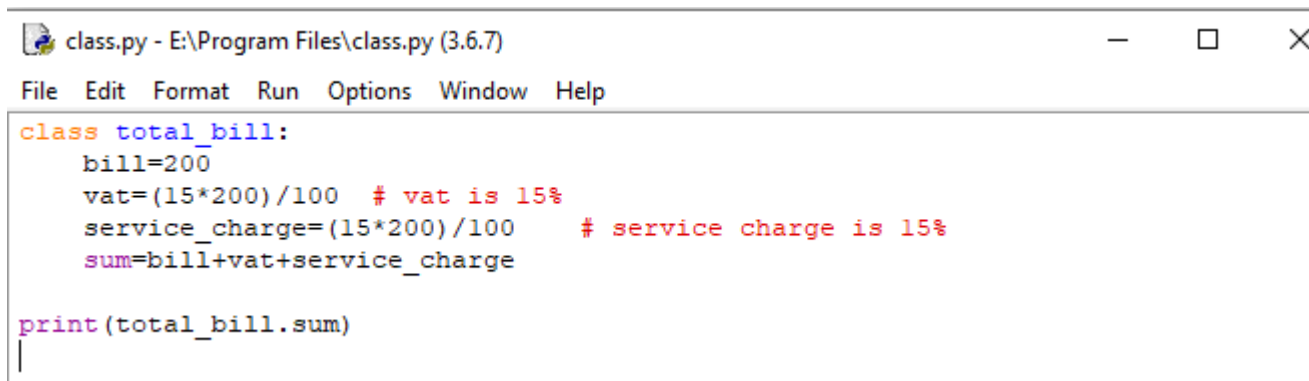
```
class MyClass:
    x = 20
num = MyClass()
print(num.x)
```

Output:



```
Python 3.6.7 (v3.6.7:6ec5cf24b7, Oct 20 2018, 13:35:33) [MSC v.1900 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/Dell/Desktop/clas.py =====
20
>>> |
```

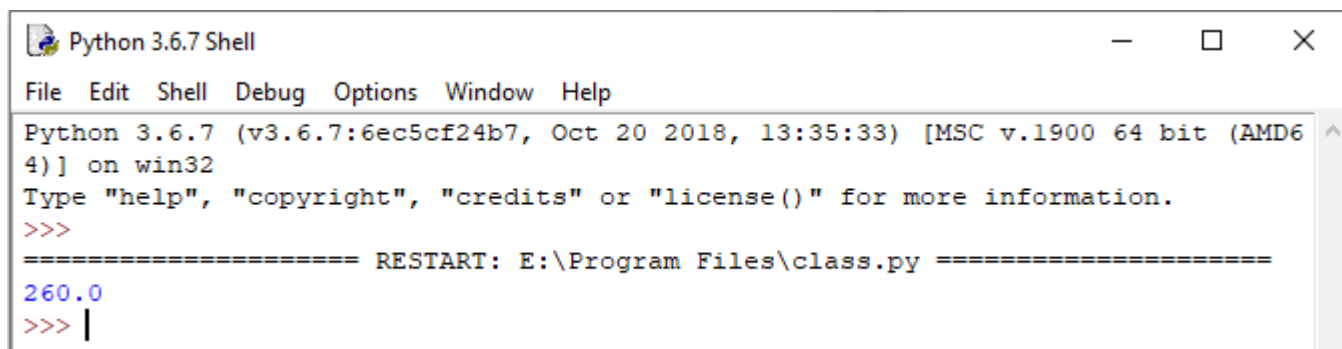
Input:



```
class total_bill:
    bill=200
    vat=(15*200)/100 # vat is 15%
    service_charge=(15*200)/100 # service charge is 15%
    sum=bill+vat+service_charge

print(total_bill.sum)
```

Output:



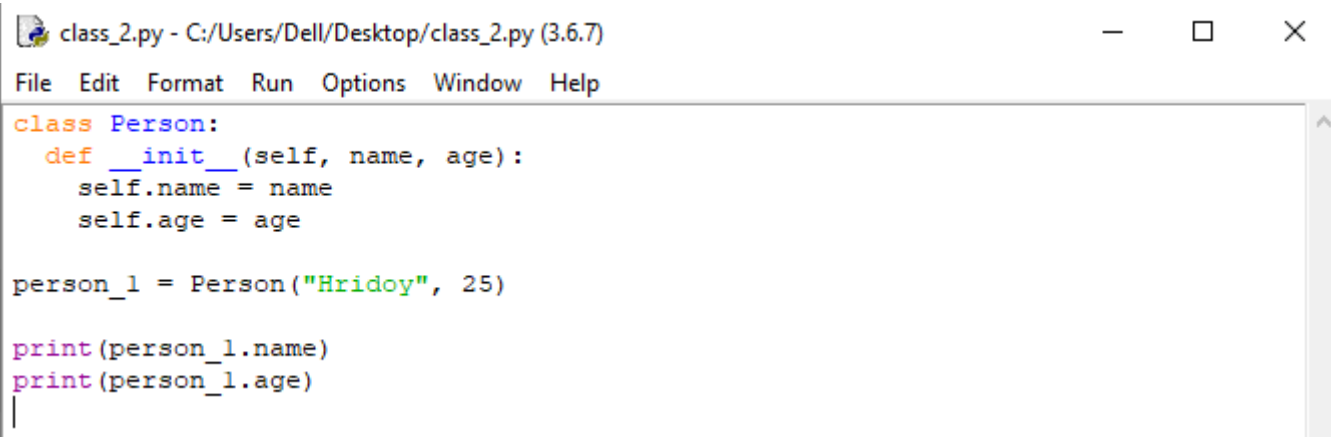
```
Python 3.6.7 Shell

File Edit Shell Debug Options Window Help

Python 3.6.7 (v3.6.7:6ec5cf24b7, Oct 20 2018, 13:35:33) [MSC v.1900 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: E:\Program Files\class.py =====
260.0
>>> |
```

The `__init__()` Function

Input:

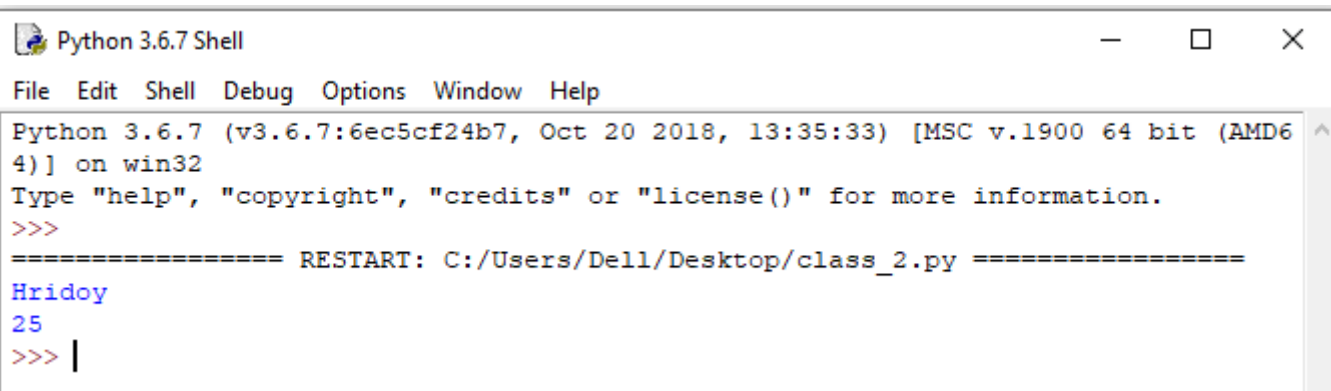


```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

person_1 = Person("Hridoy", 25)

print(person_1.name)
print(person_1.age)
```

Output:

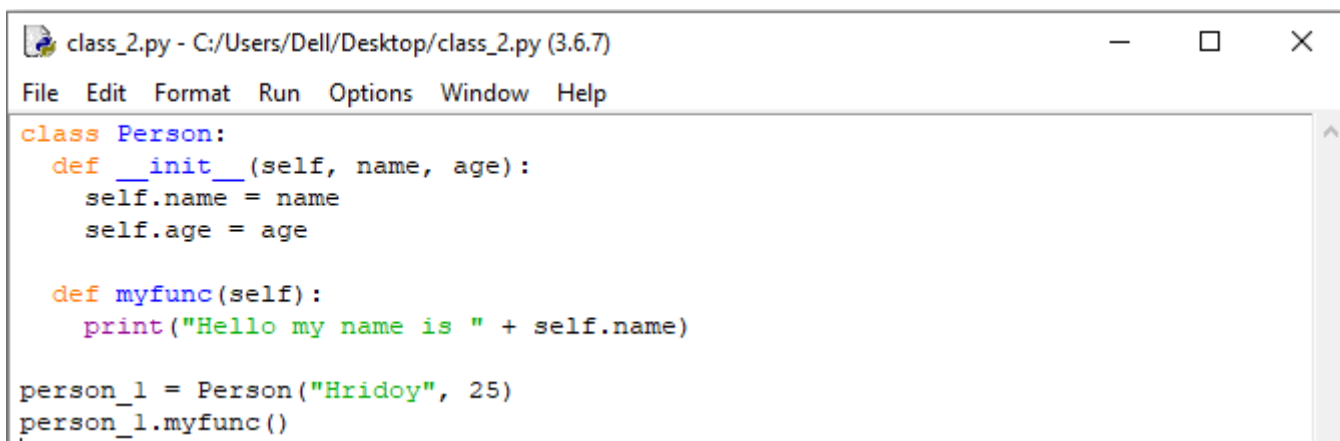


```
Python 3.6.7 Shell

File Edit Shell Debug Options Window Help

Python 3.6.7 (v3.6.7:6ec5cf24b7, Oct 20 2018, 13:35:33) [MSC v.1900 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/Dell/Desktop/class_2.py =====
Hridoy
25
>>> |
```

Input:



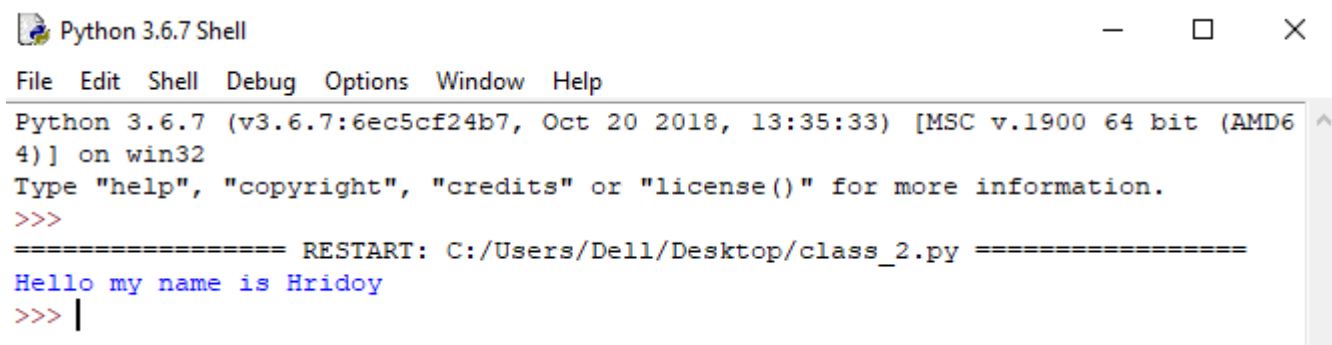
```
class_2.py - C:/Users/Dell/Desktop/class_2.py (3.6.7)
File Edit Format Run Options Window Help

class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def myfunc(self):
        print("Hello my name is " + self.name)

person_1 = Person("Hridoy", 25)
person_1.myfunc()
```

Output:



```
Python 3.6.7 Shell
File Edit Shell Debug Options Window Help

Python 3.6.7 (v3.6.7:6ec5cf24b7, Oct 20 2018, 13:35:33) [MSC v.1900 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/Dell/Desktop/class_2.py =====
Hello my name is Hridoy
>>> |
```

Input:

```
class_2.py - C:/Users/Dell/Desktop/class_2.py (3.6.7)
File Edit Format Run Options Window Help

class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def myfunc(self):
        print("Hello my name is " + self.name)

person_1 = Person("Hridoy", 25)
person_1.name='Mahmud' # Modify Object Properties
person_1.myfunc()
```

Output:

```
Python 3.6.7 Shell
File Edit Shell Debug Options Window Help

Python 3.6.7 (v3.6.7:6ec5cf24b7, Oct 20 2018, 13:35:33) [MSC v.1900 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/Dell/Desktop/class_2.py =====
Hello my name is Mahmud
>>> |
```

Input:

```
class_2.py - C:/Users/Dell/Desktop/class_2.py (3.6.7)
File Edit Format Run Options Window Help

class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def myfunc(self):
        print("Hello my name is " + self.name)

person_1 = Person("Hridoy", 25)
del person_1.age          #Delete the age property |
person_1.myfunc()
```

Output:

```
Python 3.10.2 (tags/v3.10.2:a58ebcc, Jan 17 2022, 14:12:15) [MSC v.1929 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

===== RESTART: D:\asif-office-work\python_code\class_2.py =====
Hello my name is Hridoy
|
```

Input:

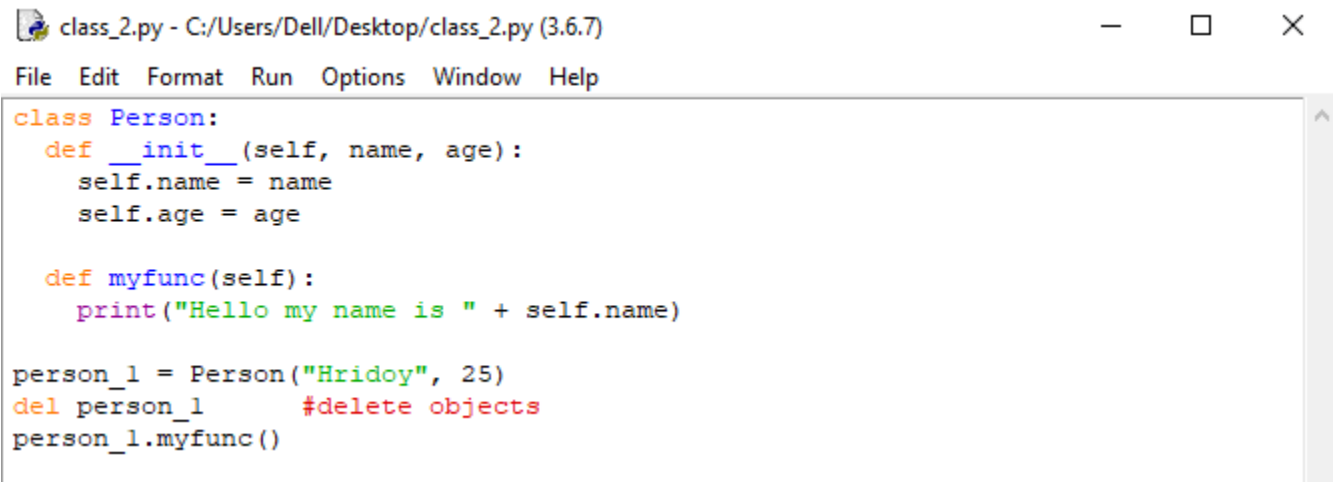
Output:

```
Python 3.10.2 (tags/v3.10.2:a58ebcc, Jan 17 2022, 14:12:15) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

===== RESTART: D:\asif-office-work\python_code\class_2.py =====
Hello my name is Hridoy

===== RESTART: D:\asif-office-work\python_code\class_2.py =====
Traceback (most recent call last):
  File "D:\asif-office-work\python_code\class_2.py", line 11, in <module>
    person_1.myfunc()
  File "D:\asif-office-work\python_code\class_2.py", line 7, in myfunc
    print("Hello my name is " + self.name)
AttributeError: 'person' object has no attribute 'name'
```

Input:

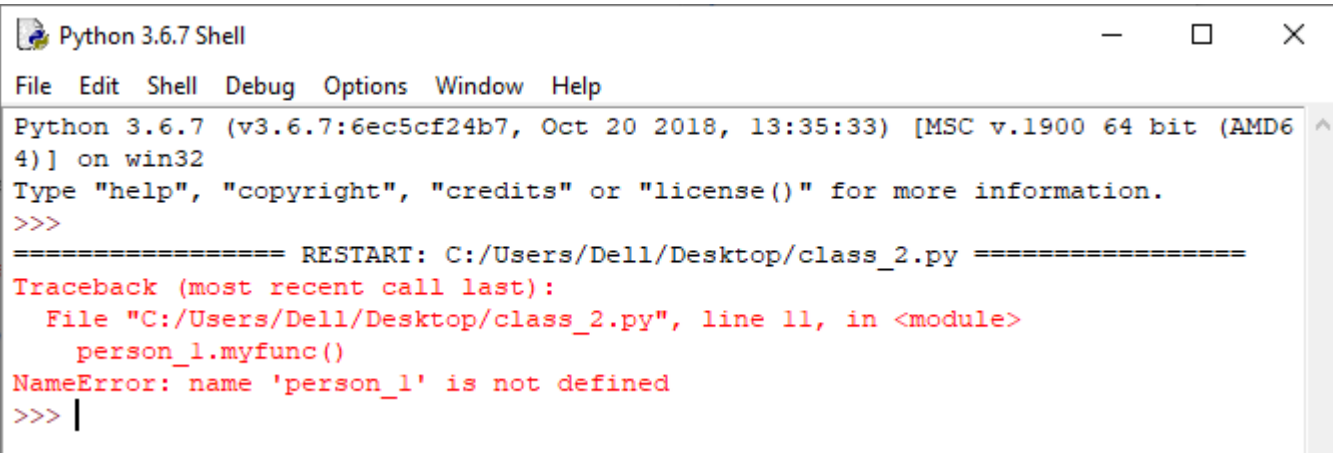


```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def myfunc(self):
        print("Hello my name is " + self.name)

person_1 = Person("Hridoy", 25)
del person_1      #delete objects
person_1.myfunc()
```

Output:



```
Python 3.6.7 Shell

File Edit Shell Debug Options Window Help

Python 3.6.7 (v3.6.7:6ec5cf24b7, Oct 20 2018, 13:35:33) [MSC v.1900 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/Dell/Desktop/class_2.py =====
Traceback (most recent call last):
  File "C:/Users/Dell/Desktop/class_2.py", line 11, in <module>
    person_1.myfunc()
NameError: name 'person_1' is not defined
>>> |
```

21.Python Inheritance:

Inheritance allows us to define a class that inherits all the methods and properties from another class.

Parent class is the class being inherited from, also called base class.

Child class is the class that inherits from another class, also called derived class.

Create a Parent Class

Any class can be a parent class, so the syntax is the same as creating any other class:

Input:

```
class Person:
    def __init__(self, fname, lname):
        self.firstname = fname
        self.lastname = lname

    def printname(self):
        print(self.firstname, self.lastname)

x = Person("Salman", "Mahmud")
x.printname()
```

Output:

```
Python 3.10.2 (tags/v3.10.2:a58ebcc, Jan 17 2022, 14:12:15) [MSC v.1929 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
```

```
===== RESTART: D:\asif-office-work\python_code\class_2.py =====
Salman Mahmud
```

Create a Child Class

```
class Person:
    def __init__(self, fname, lname):
        self.firstname = fname
        self.lastname = lname

    def print_name(self):
        print(self.firstname, self.lastname)

class child_class(Person):
    pass

x = child_class("Hridoy", "khan")
x.print_name()
```

Python 3.10.2 (tags/v3.10.2:a58ebcc, Jan 17 2022, 14:12:15) [MSC v.1929 64 bit (AMD64)] on win32

Type "help", "copyright", "credits" or "license()" for more information.

===== RESTART: D:\asif-office-work\python_code\class_2.py =====

Hridoy khan

|

22.Python Iterators:

An iterator is an object that contains a countable number of values.

An iterator is an object that can be iterated upon, meaning that you can traverse through all the values.

Technically, in Python, an iterator is an object which implements the iterator protocol, which consist of the methods `__iter__()` and `__next__()`.

Input:

```
my_str = "apple"
myit = iter(my_str)

print(next(myit))
print(next(myit))
print(next(myit))
print(next(myit))
print(next(myit))
```

Output:

```
Python 3.10.2 (tags/v3.10.2:a58ebcc, Jan 17 2022, 14:12:15) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

===== RESTART: D:\asif-office-work\python_code\class_2.py =====
a
p
p
l
e
|
```

Input:

```
my_tuple = ("apple", "mango", "jackfruit")
myit = iter(my_tuple)

print(next(myit))
print(next(myit))
print(next(myit))
|
```

Output:

```
Python 3.10.2 (tags/v3.10.2:a58ebcc, Jan 17 2022, 14:12:15) [MSC v.1929 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

===== RESTART: D:\asif-office-work\python_code\class_2.py =====
apple
mango
jackfruit
|
```

Looping Through an Iterator

Input:

```
mytuple = ("apple", "jackfruit", "mango")

for x in mytuple:
    print(x)
|
```

Output:

```
Python 3.10.2 (tags/v3.10.2:a58ebcc, Jan 17 2022, 14:12:15) [MSC v.1929 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

===== RESTART: D:\asif-office-work\python_code\class_2.py =====
apple
jackfruit
mango
|
```

23.Exception / Try Except:

The `try` block lets you test a block of code for errors.

The `except` block lets you handle the error.

The `else` block lets you execute code when there is no error.

The `finally` block lets you execute code, regardless of the result of the try- and except blocks.

The `try` block will generate an exception, because `x` is not defined:

Input:

```
try:
    print(x)
except:
    print("An exception occurred")
|
```

Output:

```
Python 3.10.2 (tags/v3.10.2:a58ebcc, Jan 17 2022, 14:12:15) [MSC v.1929 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
```

```
===== RESTART: D:\asif-office-work\python_code\class_2.py =====
An exception occurred
|
```

Print one message if the try block raises a `NameError` and another for other errors:

Input:

```
try:
    print(x)
except NameError:
    print("Variable x is not defined")
except:
    print("Something else went wrong")
|
```

Output:

```
Python 3.10.2 (tags/v3.10.2:a58ebcc, Jan 17 2022, 14:12:15) [MSC v.1929 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
```

```
===== RESTART: D:\asif-office-work\python_code\class_2.py =====
Variable x is not defined
|
```

In this example, the `try` block does not generate any error:

Input:

```
try:
    print("Hello")
except:
    print("Something went wrong")
else:
    print("Nothing went wrong")
```

Output:

```
Python 3.10.2 (tags/v3.10.2:a58ebcc, Jan 17 2022, 14:12:15) [MSC v.1929 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

===== RESTART: D:\asif-office-work\python_code\class_2.py =====
Hello
Nothing went wrong
```

Input:

```
try:
    print(x)
except:
    print("Something went wrong")
finally:
    print("The 'try except' is finished")|
```

Output:

```
Python 3.10.2 (tags/v3.10.2:a58ebcc, Jan 17 2022, 14:12:15) [MSC v.1929 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

===== RESTART: D:\asif-office-work\python_code\class_2.py =====
Something went wrong
The 'try except' is finished
|
```

Project

THE END